T3: *T*ransparent *T*racking & *T*riggering for Fine-grained Overlap of Compute & Collectives

Suchita Pati
University of Wisconsin-Madison
Advanced Micro Devices, Inc.
United States
spati@cs.wisc.edu

Shaizeen Aga
Advanced Micro Devices, Inc.
United States
shaizeen.aga@amd.com

Mahzabeen Islam
Advanced Micro Devices, Inc.
United States
mahzabeen.islam@amd.com

Nuwan Jayasena
Advanced Micro Devices, Inc.
United States
nuwan.jayasena@amd.com

Matthew D. Sinclair
University of Wisconsin-Madison
Advanced Micro Devices, Inc.
United States
sinclair@cs.wisc.edu

arXiv:2401.16677v1 [cs.AR] 30 Jan 2024

Abstract

Large Language Models increasingly rely on distributed techniques for their training and inference. These techniques require communication across devices which can reduce scaling efficiency as the number of devices increases. While some distributed techniques can overlap, and thus, hide this communication with independent computations, techniques such as Tensor Parallelism (TP) inherently serialize communication with model execution. One approach to hide this serialized communication is to interleave it with the producer operation (of the communicated data) in a *fine-grained* manner. However, this fine-grained interleaving of communication and computation in software can be difficult. Furthermore, as with any concurrent execution, it requires compute and memory resources to be shared between computation and communication, causing resource contention that reduces overlapping efficacy.

To overcome these challenges, we propose T3 which applies hardware-software co-design to transparently overlap serialized communication while minimizing resource contention with compute. T3 *transparently fuses* producer operations with the subsequent communication via a simple configuration of the producer's output address space and requires minor software changes. At the hardware level, T3 adds a lightweight *track and trigger* mechanism to orchestrate the producer's compute, and communication. It further uses *compute-enhanced memories* for communication's attendant compute. As a result, T3 reduces resource contention, and efficiently overlaps serialized communication with computation. For important Transformer models like T-NLG, T3 speeds up communication-heavy sublayers by 30% geomean (max 47%) and reduces data movement by 22% geomean (max 36%). Furthermore, T3's benefits persist as models scale: geomean 29% for sublayers in ~500-billion parameter models, PALM and MT-NLG.

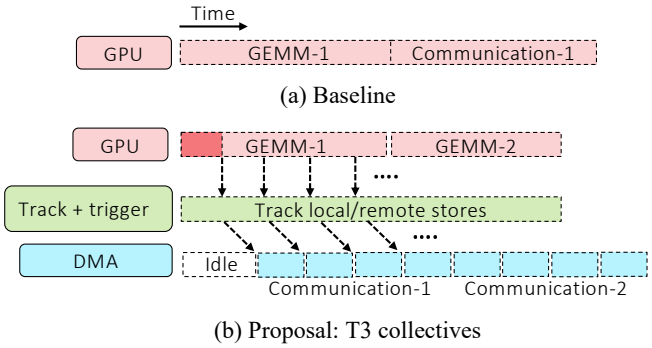


Figure 1. T3 overview.

CCS Concepts: • **Computer systems organization** → *Single instruction, multiple data*; *Neural networks*; • **Computing methodologies** → *Parallel computing methodologies*; *Distributed computing methodologies.*

Keywords: Distributed Machine Learning, Collective Communication, Transformers, GPUs, Fusion, Fine-grained Overlap, Near-memory Computing

ACM Reference Format:
Suchita Pati, Shaizeen Aga, Mahzabeen Islam, Nuwan Jayasena, and Matthew D. Sinclair. 2024. T3: *T*ransparent *T*racking & *T*riggering for Fine-grained Overlap of Compute & Collectives. In *29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '24), April 27-May 1, 2024, La Jolla, CA, USA.* ACM, New York, NY, USA, 19 pages.
https://doi.org/10.1145/3620665.3640410

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).
ASPLOS '24, April 27-May 1, 2024, La Jolla, CA, USA
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-0385-0/24/04.
https://doi.org/10.1145/3620665.3640410

1 Introduction

Deep neural networks (DNNs) have transformed society with significant accuracy improvements for tasks including speech recognition [87], image classification [22, 37, 40, 79, 82, 83], machine translation [21], autonomous agents [41], language processing [14, 68] and text generation [9]. This tremendous, transformative effect has been enabled by a virtuous synergy of (1) better hardware systems, (2) larger datasets, and (3) improved ML structures and algorithms that further benefit from more efficient hardware and larger datasets. This is especially true for Transformers, which have become popular for a wide range of tasks [8] and have shown considerable strides in artificial general intelligence [73]. Transformers have had exponential growth in datasets and model size: parameters have increased from 340 million in BERT [14] to 540 billion in PALM [12]. Accordingly, their memory and computational demands have also increased, making them increasingly reliant on distributed techniques: multiple accelerators (e.g., GPUs) pooling their collective memory capacities and compute capabilities to collaboratively execute a DNN. However, the resulting communication between devices has become a significant proportion of their execution time and has limited the scaling efficiency with increasing device count [36, 49, 64].

Transformers frequently use two key distributed techniques in conjunction: data parallelism (DP) and model parallelism (MP). DP parallelizes training by partitioning the dataset and replicating the model across devices, requiring communication and aggregation (*all-reduce*) of gradients. Conversely, MP partitions large models that cannot fit in a single device's memory. Tensor-parallelism (TP), a type of MP, requires an all-reduce of layer outputs between devices as well. Among these distributed techniques, TP's communication typically lies on the critical path of model execution, as shown in Figure 1(a) and can be a significant proportion of runtime (~45% [64]), resulting in a sub-linear increase in throughput as the number of devices increases.

While some prior works have sped up communication by up to 2× with *in-network computation*, they are topology-dependent (requiring switches) and further, cannot eliminate serialized communication from the critical path [36]. Distributed techniques with abundant coarse-grained independent compute (e.g., DP) often overlap (and hide) communication with independent computations to improve efficiency. Although serialized communication scenarios also offer such potential, they require a fine-grained overlap of computation and communication, which presents its own challenges. Enabling their fine-grained overlap in current systems either requires expensive fine-grained synchronization [25] or changes to matrix multiplication (GEMMs) kernels which can be disruptive to GPU software infrastructure [86] (Section 3.1). Furthermore, overlapped compute and communication contend for both compute units and memory bandwidth, reducing overlap's efficacy [25, 86] (Section 3.2). Prior approaches that reduce contention only address coarse-grained overlap of compute and communication in cases like DP and lack support for fine-grained overlap in serialized collectives [71]. Moreover, they rely on dedicated accelerators. *Therefore, no existing technique achieves a transparent overlap of serialized communication with computation while minimizing resource contention.*

To overcome these, we propose T3 (Figure 1(b)). T3 *transparently* fuses producer operations with the subsequent communication by *configuring the producer's output address space* to initiate communication directly on the producer's store, requiring minimal application changes. It uses a lightweight and programmable hardware *tracker* to track the producer/-communication progress and *triggers* communication using pre-programmed DMA commands, requiring no additional GPU compute resources for communication. Furthermore, to reduce contention for memory bandwidth between the producer and communication, T3 leverages recently proposed compute-enhanced memories [34, 38] to atomically update memory on stores, thus reducing memory traffic due to communication-related reductions. Finally, T3 employs a simple yet effective arbitration policy between the producer and communication memory streams to minimize any remaining contention. Overall, T3 transparently overlaps serialized communication with minimal resource contention. This improves compute and network utilization, and in turn, can enable better throughput scaling with increasing device count. We make the following key contributions:

- We propose T3 which enables fine-grained overlap of serialized communication with its producer computation whilst lowering application impact and managing compute and memory interference.
- To manage application impact, T3 configures the producer's output address space mapping to initiate communication on stores, requiring minor modifications to the producer kernels.
- To manage compute resources contention, T3 uses a lightweight programmable tracker that tracks producer progress and triggers communication using existing DMA engines requiring no additional compute resources.
- Finally, to tackle memory bandwidth contention between computation and communication, T3 harnesses emerging near-memory compute technology to reduce data movement due to communication. Further, T3 also devices a simple yet effective memory controller arbitration policy to better interleave computation and communication memory traffic.
- Similar to prior work [32], we extend Accel-Sim [33] to accurately model multi-GPU systems (6% error). Our results show that T3 speeds up sliced Transformer

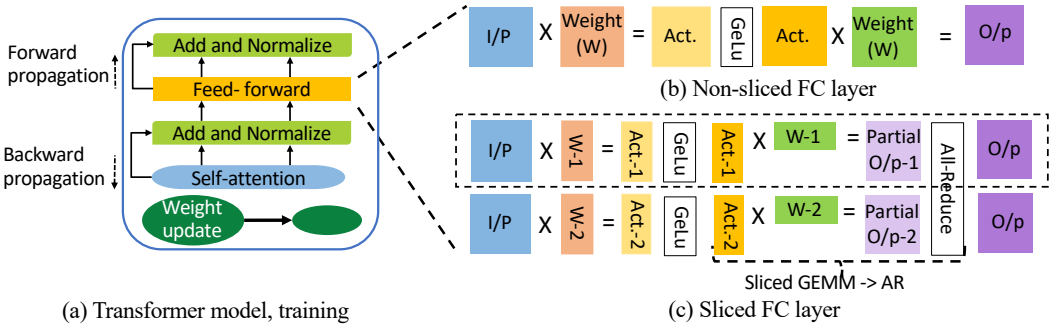


Figure 2. (a) Transformer (b) Fully-connected (FC) layer (c) Tensor-sliced FC layer with all-Reduce on the critical path.

sub-layers from models like Mega-GPT-2 [78] and T-NLG [47] by 30% geomean (max 47%) and reduces data movement by 22% geomean (max 36%). Furthermore, T3's benefits persist as models scale: geomean 29% for sublayers in ~500-billion parameter models, PALM and MT-NLG. Overall, T3 speeds up model training by up to 12% and inference (prompt phase) by up to 15%.

2 Background & Motivation

2.1 Transformers & Need for Distributed Computing

Transformers [84] have become the general-purpose architecture for a wide range of tasks/domains (e.g., for text, image) [8]. Models use the Transformer *encoder* or *decoder* as their basic building block, each with an attention sub-layer and a fully connected (FC) sub-layer (as shown in Figure 2(a)) which manifest as matrix multiplication operations (GEMMs). Each layer also contains a few residual connections and layer normalizations which manifest as element-wise operations, and are often fused [16, 18, 81, 85] with the GEMMs. As shown in Figure 2(b), these GEMMs entail multiplication of layers' weight matrices by an input matrix (with each vector representing an input token). During training, the input matrices contain multiple tokens from one or more (if batched) input sequence(s). During inference, there are two execution phases: a *prompt* phase to process all tokens in the input sequence(s) and a *token generation* phase to iteratively process and generate one token at a time for each input sequence [62]. The prompt phase operations are similar to those in training, while the generation phase has GEMMs with small input matrices or matrix-vector operations (GEMVs) if there is no batching.

Most Transformer models' memory capacity requirements exceed a single device. Thus, they employ distributed techniques and use multiple accelerators (e.g., GPUs) collaboratively. Furthermore, the aggregate computational capacity of multiple devices also accelerates training by enabling the processing of large input datasets in parallel. Thus, since Transformers and their datasets (usually large corpora of unlabeled text) have increased by several orders of magnitude in size, distributed techniques are often mandatory and increasingly require many devices. This scaling will only increase for future models.

2.2 Distributed Techniques & Associated Collectives

Transformers employ many distributed techniques, each with associated *communication* between devices. Data parallelism (DP) trains model replicas on multiple devices, each on a disjoint set of data, and requires a reduction of gradients every iteration. Tensor parallelism (TP) [78] and pipeline parallelism (e.g., GPipe) [23] slice the model across multiple devices. While the former slices each layer requiring activation reduction, the latter partitions the model layer-wise requiring peer-to-peer transfer of activations. ZeRO-based optimizations [70] also slice model weights or offload them to slower but larger (e.g., CPU) memories, and require them to be gathered before layer executions. Finally expert parallelism [35] partitions mixture-of-expert (MoE) models [17, 69] such that each device hosts a single expert and requires exchange of input data based on input-to-expert mapping. These communication patterns are handled by *collectives* such as *reduce-scatter*, *all-reduce*, *all-gather*, *all-to-all*. While most of this communication can be hidden by independent compute operations [49, 63, 64], albeit with some resource contention [36, 71], the all-reduce in TP is not (detailed in Section 2.4). Thus, we focus on all-reduce in TP and discuss other techniques/collectives in Sections 7.1 and 7.2.

2.3 All-Reduce & Ring Implementations

The all-reduce (AR) collective reduces (element-wise sums) arrays from each of the devices. Although there are multiple implementations of AR, one of the most bandwidth-efficient, and thus most commonly used, implementations is *ring-AR*. Ring-AR consists of a ring reduce-scatter (ring-RS) followed by a ring all-gather (ring-AG). As shown in Figure 3, ring-RS is done in multiple steps. The arrays are chunked on each device, and during each step, all devices send their copy of a *unique* chunk to their neighbor in the ring. The devices then

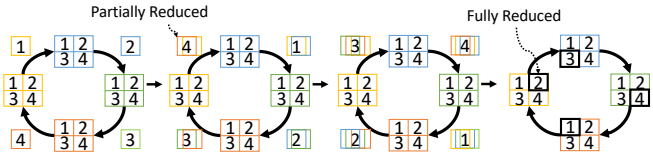


Figure 3. Ring implementation of reduce-scatter collective.

reduce their local copy of the chunk with the received copy and forward it to their neighbor in the next step. With N devices and the array chunked N ways, this process requires $N-1$ steps until each device has a completely reduced copy of one chunk. Ring-AG is similar but does not have reductions; it also requires $N-1$ steps until each device has all the reduced chunks. In the remainder of the paper, we use AR, RS, and AG to refer to their ring implementations and discuss other implementations in Section 7.1.

2.4 All-Reduce is on the Critical Path & can be Large

Transformers require tensor parallelism (TP) [78] to increase the aggregate memory capacity available to them. However, it requires ARs on the critical path (between layers). Figures 2(b) and 2(c) show the FC sub-layer's original operations versus the operations when sliced across two devices (TP=2 in Figure 2(c)). Each device (dotted box) only has a slice of the weights. Since the GEMM corresponding to the second sliced weight only generates a partial output, it requires an AR before the next layer executes (highlighted by "Sliced GEMM→AR"). These GEMM and AR operations execute as separate kernels and are serialized.

These serialized ARs can become a bottleneck. Figure 4 shows the execution time breakdown of Transformers between "Sliced GEMM→AR" and other operations for multiple current and futuristic Transformers (setup detailed in Section 5.1.2, 5.2). For large models (e.g., Mega-GPT-2, T-NLG) we consider 8- and 16-device TP. For very large models (e.g., PALM, MT-NLG) we consider 32-way slicing, and for futuristic ones with one and ten trillion parameters, we consider 64-way sharding. The increasing TP slicing is necessary because these models' larger sizes cannot fit in 16 GPUs [64] and the increased slicing is also enabled by nodes with larger device counts [59, 86]. Like prior work [36, 49, 64], we find that communication is a considerable fraction of the overall runtime: Megatron-GPT-2 (Mega-GPT-2) and T-NLG spend up to 34% and 43% of their training and inference (prompt phase) time on communication. These trends also hold for the very large and futuristic Transformers: communication can be up to 46% and 44% of their runtime, respectively. Additionally, since compute FLOPS scales much more than network bandwidth [19], these proportions will only increase in the future. For example, if the GEMMs become 2× faster, communication increases to 75% of model execution time – making scaling to multiple devices extremely inefficient and potentially leaving GPUs idle while communication happens.

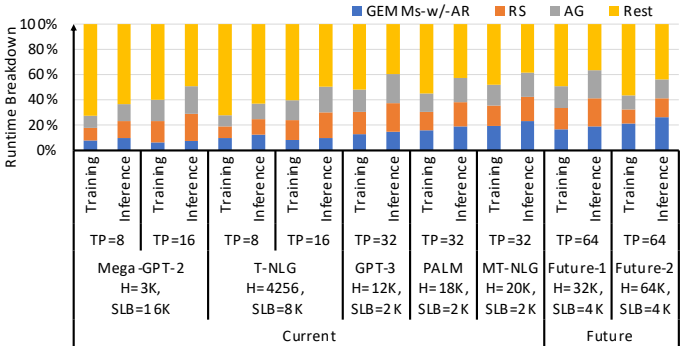


Figure 4. Transformer time spent on reduce-scatter (RS) and all-gather (AG) collectives as well as GEMMs which require collectives.

Thus, addressing serialized AR is critical to Transformer scaling.

2.5 Enabling Compute-Communication Overlap

Overlapping collective kernels with independent compute kernels has been key to scaling DNNs in other distributed approaches (e.g., DP, GPipe [23]). While TP does not have independent kernels to overlap AR with, we observe that it can benefit from a *fine-grained overlap* with the producer GEMM itself. Transformer GEMMs have large outputs, which are tiled/blocked and require many GPU workgroups (WGs) to complete. Consequently, a GEMM cannot always execute all its WGs concurrently on the limited number of GPU compute units (CUs). Thus, a GEMM executes and generates output in multiple *stages*, where each stage is a set of WGs that the CUs can accommodate. This holds even for sliced GEMMs that require AR. As shown in Figure 5, GEMMs in TP are sliced in the K (or dot-product) dimension which decreases compute per WG, but the output size, WG count, and WG stages remain the same. We utilize this observation to enable fine-grained overlap: communication of one stage's output data can be overlapped with compute of the next stage. However, achieving practical and efficient fine-grained overlap is challenging as we describe in Section 3.

3 Challenges With Fine-grained Compute-Communication Overlap

This section details key challenges with the fine-grained overlap of compute and communication.

3.1 Complex & Expensive to Implement in Software

The producer and collective operations execute as separate kernels on GPUs; the producer (GEMM) generates the data, after which the collective orchestrates their bulk communication and reduction. Extending the software for their fine-grained interleaving can be complex and expensive. It would involve breaking the producer and collective into smaller kernels or using dynamic parallelism, both of which can

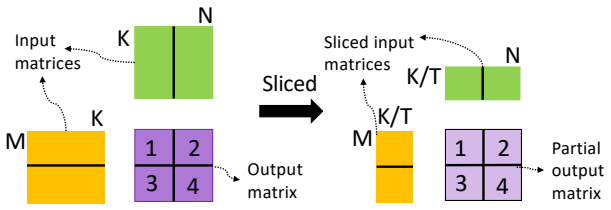


Figure 5. GEMM (left) when sliced in the dot-product dimension (right) still generates the same number of data blocks.

increase launch overheads and synchronization costs. Alternatively, it can be achieved by writing fused GEMM and collective kernels, but this can incur significant programming effort [16, 18, 81, 85]. First, BLAS libraries have hundreds of GEMM kernels optimized for different input sizes and GPU architecture, generated via an expensive tuning process [3]. Second, collectives are also of different types, and each has implementations optimized for different topologies. Creating fused kernels for every combination of GEMM and collective implementations can thus be extremely complex and expensive. Hence, it is imperative to achieve a fine-grained overlap of compute and communication without altering GEMM implementations.

3.2 Resource Contention Between Producer & Collective

Overlapped GEMM and AR contend for GPU resources, specifically compute units (CUs) and memory bandwidth, which slow down overall execution.

3.2.1 Compute Sharing. Concurrently executing GEMM and AR kernels must share CUs and their components including L1 cache, LDS, and vector registers. This contention may affect their performance relative to their isolated execution. Figure 6 evaluates the impact of concurrently executing GEMM and AR using our setup in Section 5.1.1 and Table 1. Specifically, Figure 6 shows the (normalized) GEMM and AR time for Mega-GPT-2 and T-NLG (with TP=8) sub-layers (Attn. and FC-2) when run in isolation with varying CU count splits (e.g., the 72-8 bars show GEMM's isolated execution time with 72 CUs and AR's with eight CUs). For each case, it also shows *potential-overlap-speedup*, the speedup overlapping AR and GEMM can obtain versus sequentially executing GEMM and AR when each has all 80 CUs. We calculate the overlapped time as max(GEMM time, AR time). The ideal case assumes no sharing impact: the GEMM has all the 80 CUs and the AR is fast but free (evaluated by running it with all 80 CUs in isolation). As a result, the ideal case has the maximum potential overlap speedup of 1.67× geomean. However, AR slows down considerably (geomean ~41% slowdown) when allocated only eight CUs (72-8 case) compared to when it had all CUs. This significantly decreases

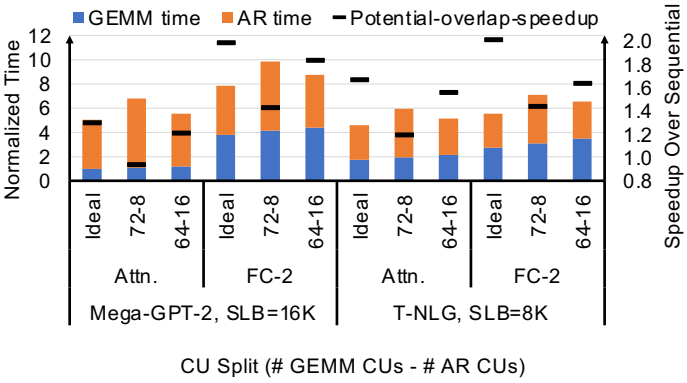


Figure 6. Evaluating how the benefits of overlapping GEMM and RS, across model layers, are impacted by compute unit (CU) sharing. The X-axis shows how CUs are split between GEMM and AR, using the GPU setup from Table 1, in the format *A-B*. *A* represents the number of CUs the GEMM uses, while *B* represents the number of CUs AR uses. Ideal assumes no sharing, the GEMM has all CUs, and AR is free.

the potential-overlap-speedup to 1.18× geomean. While AR performance improves with 16 CUs (only ~7% slowdown in 64-16 case), GEMMs slow down (geomean ~21% slowdown) since they now only have 64 CUs. Overall, while better than the 72-8 case, potential speedups fall short (1.49× geomean) compared to the ideal case. Moreover, this assumes no contention due to memory bandwidth sharing (discussed next) and thus underestimates slowdowns. Overall, sharing of CUs reduces overlapping efficacy and it is crucial to preserve the compute resources dedicated to GEMMs.

3.2.2 Memory Bandwidth Sharing. GEMM and AR kernels also compete for memory bandwidth when run concurrently. As shown in Figure 3, at each step AR kernels a) read an array chunk from memory to send it to one neighbor GPU and also b) write to memory the chunk it received from another neighbor. Reduce-scatter (RS) additionally requires a memory read of the local chunk for reduction. Moreover, the memory traffic due to AR communication can be bursty. This additional, bursty memory traffic due to AR can slow down critical memory accesses by the producer GEMM, with the impact higher for GEMMs for which inputs do not fit in GPU's last level cache (LLC) as we will show in our evaluation in Section 6.1.2 and Figure 17. Thus, to enhance overlap efficiency, it is essential to limit memory traffic due to communication and/or limit their contention with GEMM.

Prior work also studied contention between communication and computation [71], albeit in DP setups with coarse-grained GEMM and AR overlap. They show that AR slows down by up to 2.4× when run concurrently with GEMMs, and the slowdown is even higher when run concurrently with memory-intensive embedding lookups in recommendation models. For TP, they observe a 1.4× slowdown when executed concurrently with GEMMs.

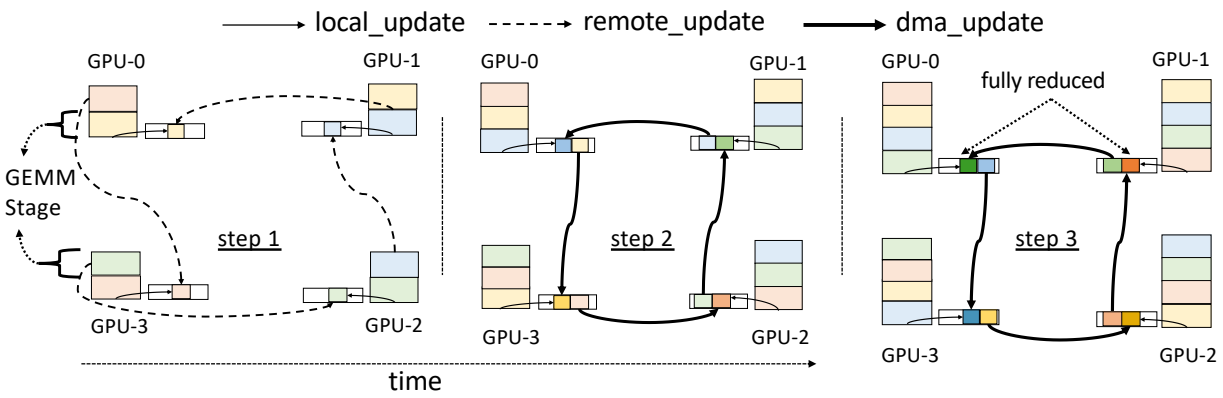


Figure 7. Overview of fused GEMM and ring reduce-scatter with T3 on a four-GPU node.

4 T3: Transparent Tracking & Triggering

To overcome the aforementioned challenges of complex software and resource contention with fine-grained overlap of compute and communication, we propose T3.

4.1 T3 Overview

Modern GPUs first execute the producer GEMMs and store their outputs in their local memory. Afterwards they initiate the collective operation (Section 3). T3 instead initiates the collective immediately as GEMMs generate data to enable fine-grained overlap. It uses a ***track & trigger*** mechanism to monitor GEMM's/collective's progress and to orchestrate communication, requiring no additional CUs (Section 4.2). It leverages ***near-memory compute*** for reductions to reduce memory traffic due to communication (Section 4.3). Finally, it does these ***transparently***, with minor kernel modifications (Section 4.4).

Figure 7 illustrates a four-device reduce-scatter (RS) overlapped with its producer GEMM. This GEMM executes in multiple *stages* of WGs dictated by its input and kernel implementation (Section 2.5), while RS executes in multiple *steps* dictated by the number of devices involved (Section 2.3). For simplicity of illustration, we show the number of GEMM stages to be one more than the number of required ring steps. In each step, a GEMM stage's execution and reduction of its output happen in parallel to the communication of the previous stage output. In the first step, the output is communicated to remote devices directly by the GEMM (*remote_update*). The later, *steady state*, steps require a DMA (*dma_update*). For N devices, this steady state step is performed $N-2$ times, on different chunks. Focusing on GPU-0 in the steady state, step-2, as shown in Figures 7, the GPU executes/generates output for GEMM stage-3 while also receiving (via DMA) a copy of stage'3 output (blue) from its neighbor, GPU-1. This occurs in parallel to GPU-0's DMA of the reduced copy of GEMM stage-2 data (yellow) to GPU-3, thus overlapping communication. T3 leverages *near-memory computing* (NMC) to atomically update memory locations on these local and DMA updates, resulting in a partially reduced copy of the stage-3 chunk without requiring additional reads or GPU CUs (Section 4.3). Once they complete, GPU-0 initiates a *dma_update* of the chunk to its neighbor's (GPU-3) memory as shown in step-3. This automatic tracking of updates and DMA triggering is done using a lightweight and programmable hardware Tracker, further reducing dependency on GPU CUs (Section 4.2). These remote / DMA updates are done transparently by configuring the GEMM's output address mapping, with minor application and kernel modifications (Section 4.4).

We also make minor runtime and hardware changes to improve T3's performance. To enable the perfect overlap of GEMM and RS in Figure 7, we stagger the scheduling of GEMM workgroups (WGs) across GPUs (Section 4.4). Moreover, we also augment the memory system with a simple yet effective memory controller arbitration (MCA) policy to manage memory contention between compute and communication (Section 4.5).

Figure 8 shows a GPU with T3's enhancements (in orange) executing the steady state step described above. The GPU executes the GEMM to generate local updates for a stage (L1). Concurrently the GPU receives DMA updates for the same stage (D1a) and sends DMA updates for the previous stage (D1b). At the memory controller, the modified *MCA* arbitrates between the local and DMA traffic to prevent contention. Following this, the updates are sent to *NMC-enhanced DRAM* (L2a,D2a) while the *Tracker* is updated with their progress (L2b,D2b). Once the Tracker observes the required local and DMA updates to a memory region, it triggers their DMA transfer to the neighbor GPU (L3).

We use the 4-GPU GEMM-RS overlap as a running example to describe T3. RS is more challenging to overlap due to reductions and extra memory traffic. Further, the ring

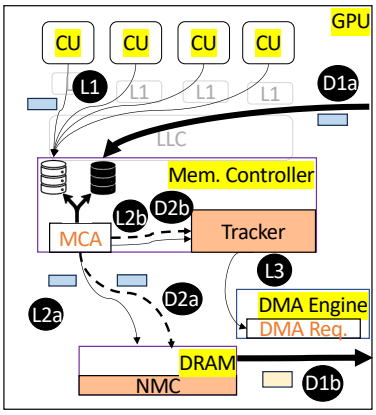


Figure 8. GPU with highlighted T3 enhancements (in orange) executing a steady-state fused GEMM-RS step.

configuration is more complex than others. Thus, we detail T3 using ring-RS and discuss additional collectives in Section 7.1.

4.2 T3 Tracking & Triggering

T3's *programmable track & trigger* mechanism is key to transparently enabling fine-grained overlap of producer and collective without using compute resources. As shown in Figure 9, T3 automatically transfers copies of data between devices when ready (e.g., in Figure 7, T3 triggers DMA update of stage-2 data from GPU-0 to GPU-3 once both GPU-0's local and GPU-1's remote updates are complete). This is enabled by a lightweight *Tracker* at the memory controller, that tracks local and remote/DMA accesses to memory regions and triggers a DMA transfer once the required accesses are complete. Since the condition when a DMA is triggered (e.g., number of remote and local updates) and DMA transfer details (e.g., addresses, operation type) vary per collective type and implementation, they are programmed ahead of time using address space configuration (detailed in Section 4.4 and Figure 12).

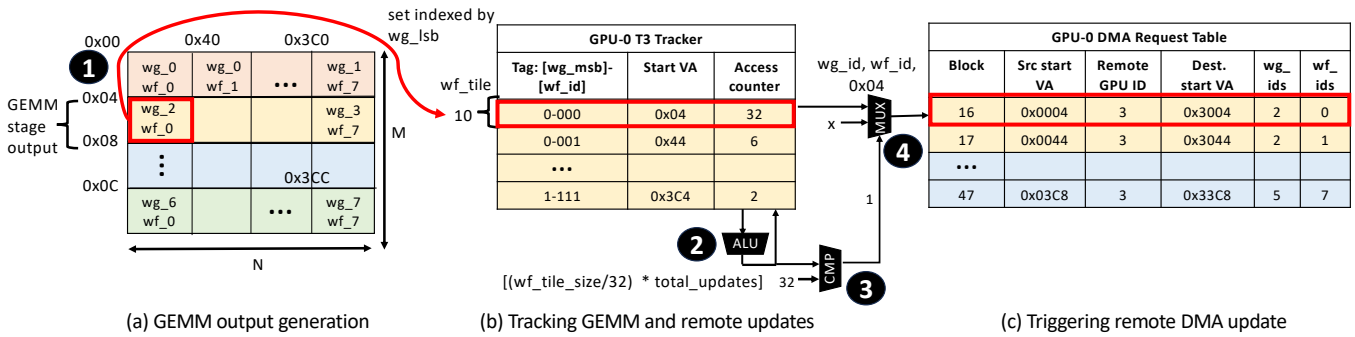
4.2.1 Tracker. The Tracker tracks both local and remote memory updates of a GEMM stage and triggers its DMA. As shown in Figure 9(a) and (b), it does so at wavefront (WF, i.e., a group of threads that execute in lockstep) granularity ① – i.e., the Tracker tracks the memory region a WF updates. This assumes tiled GEMM implementations and that each WF/WG generates a complete tile of data, as is the case in all evaluated GEMMs [30, 61]. However, T3 can also handle other implementation (Section 7.7). An update increments the counter at its corresponding WF's (wf_id) Tracker entry ②. This is done by all local, remote, and DMA updates that arrive at the GPU's memory controller (e.g., GPU-0 does not track GEMM stage-1 as its WFs neither write locally nor are its remote updates received). The incremented counter value is checked for a maximum threshold, which is set to the product of WF output size (wf_tile_size) and the total updates expected per element ③. The wf_tile_size is determined by the GPU driver using the output size and WF count ($(M * N)/\#WF$). The total updates expected per element for ring-RS is two but changes with collective type/implementation and is thus configurable (detailed in Section 4.4). Once the threshold is reached, the final write triggers the DMA (④ in Figure 9(c) and detailed in Section 4.2.2). The Tracker is checked once the accesses are enqueued in the memory controller queue (MCQ) and thus are not in the critical path.

WF-based tracking is beneficial as a producer's (or GEMM's) stage may not update contiguous memory regions. As shown in Figure 9(a) this can happen due to column-major allocation of arrays in BLAS libraries [2, 61] and row-major scheduling. This makes address-based tracking expensive (requires storing several addresses or complex table indexing functions) which WF-based tracking avoids. The Tracker has a total of 256 entries, indexed using the workgroup (WG) ID's LSBs, wg_lsb (8 bits). Each entry is set associative and is tagged using wg_msb, wf_id. wg_msb is $log_2(maxWGsperstage/256)$ bits and wf_id is three bits for a maximum of eight WFs per WG. We set the maximum entries based on the maximum WGs possible in a producer stage. Each entry has a starting virtual address (smallest address per WF), and an accesses counter, making the Tracker size 19KB. The tracking additionally requires the source wg_id and wf_id as metadata in memory accesses and forwarding of their virtual addresses to the memory controller (to trigger the DMA in Section 4.2.2).

4.2.2 Triggering DMA. Once the required accesses to a WF's memory region are issued, T3 DMAs the data to the remote GPU (④ in Figure 9(c)). As shown in Figure 9(c), the DMA commands are pre-programmed by the GPU driver and are configurable (detailed in Section 4.4 and Figure 12) as the DMA regions/operations can differ based on the collective type and implementation. The granularity of the DMA block/table entry is set to be equal to or larger than the Tracker granularity (wf_tile). The memory access which completes the required accesses at the Tracker entry (Section 4.2.1) marks the corresponding DMA entry ready and also populates it with the wg_id and wf_id which are required by the destination GPU's Tracker. If DMA blocks are a multiple of wf_tile, an additional counter per DMA entry can track their completion. Using the pre-programmed starting source/destination virtual address, wf_tile_size, and the output dimensions (M, N), the DMA engine dynamically generates the remaining virtual addresses to initiate the DMA.

4.3 Near-Memory Reductions

To perform reductions on producer and DMA updates without occupying GPU compute resources, T3 leverages compute-enhanced memories. We assume an HBM-based DRAM architecture with near-memory op-and-store support as has been



(a) GEMM output generation (b) Tracking GEMM and remote updates (c) Triggering remote DMA update

Figure 9. T3 Track & Trigger.

proposed by recent works [52, 66]. We envision such compute support to be implemented via ALUs near DRAM banks as has recently been proposed by memory vendors [34, 38]. However, T3 can also leverage other reduction substrates (Section 7.4).

T3 leverages this near-memory computing (NMC) capability to enable GEMM stores and DMA transfers to directly update and reduce copies of data, when required by collectives. For DMA transfers, the operation type (store vs. updates) is directly specified in the command (address space configuration in Figure 12 and Section 4.4). For GEMMs, we utilize two flags. First, we use an "uncached" flag during memory allocation to ensure that the output is not cached in any GPU's caches (such allocations are supported in existing GPUs). Thus, writes are directly sent to DRAM which acts as the point of aggregation for all (local, remote, DMA) updates. The queuing of updates in the memory controller queue guarantees their atomicity; at any given time, only a single instruction can be issued and executed by near-bank ALUs. Second, we use an "update" flag in the GEMM API call to enable stores of the GEMM to update the DRAM. The "update" flag is sent (via kernel packets [4]) to the CUs to tag the kernel's stores with one-bit "update" info (similar to prior work [27, 28, 65]). These are processed by the memory controller to generate the op-and-store commands.

In addition to freeing up CUs for GEMMs, NMC helps reduce memory traffic due to communication. Figure 10 shows memory accesses in a steady-state RS step in baseline and with T3. In baseline RS, CUs read two copies of data (local copy, and received copy from the previous neighbor) and write the reduced data to the next neighbor's memory. T3 only requires one read of the data to DMA update the neighbor GPU memory using NMC. Overall, T3 with NMC reduces the dependence on GPU CUs and further reduces (or eliminates, *direct-RS* in Section 7.1) data movement required for communication.

4.4 Configuring Producer's Output Address Space

Modifying producer kernels, especially for many GEMMs of different shapes and sizes, to fuse and overlap collectives, can be impractical (Section 3.1). T3 avoids this by configuring the producer's output address space mapping which is used to program the Tracker and DMA commands. Figures 11 and 12 show this configuration for GPU-0 from the fused GEMM-RS example in Figure 7.

Since there are four devices, GEMM's output array is chunked four ways. In GPU-0, the GEMM writes its stage-1 output directly to GPU-3's memory (step-1 in Figure 7), while its stage-2 and stage-3 output is first written to local memory and later DMA'd to GPU-3 (stage-4 is only written locally once and is not DMA'd). Thus, GPU-0 requires memory mappings of these chunks with that of GPU-3 as shown in Figure 11. This configuration differs per collective type and topology-optimized implementation (see Section 7.1) and, similar to modern collective implementations, can be predefined in collective libraries [1, 55]. Figure 12 shows an example of this using pseudo-code.

The configuration in Figure 12 defines this mapping for the GEMM output using two different API calls: *remote_map* and *dma_map*. *remote_map* is used for fine-grained remote writes/updates (for stage-1), which uses existing GPU support for peer-to-peer load/store by threads [57]. Conversely, *dma_map* is used for coarse-grained DMA writes/updates (for stage-2,3) which leverages existing support for memory copies by DMA engines in GPUs (DirectGMA and others [50, 51, 57]). A *dma_map* call also defines the DMA functionality (store vs. update), and its triggering condition (number of stores/updates per element). It can also be extended to specify granularity (wf_tiles per DMA block in Figure 9(c)). These calls are used to pre-program the Tracker and DMA commands to enable automatic communication of data when ready (Section 4.2).

Fusion in ring-based collectives also benefits from producers (on different devices) generating data chunks in a staggered manner. In Figure 7, GPUs stagger the generated data by one stage; in step-1, GPU-0 executes stage-1, while GPU-1 executes stage-2, and so forth. This is enabled by staggering WG scheduling across devices. Alternatively, it can also be enabled by fetching appropriate implementation from BLAS libraries with staggered output tile-to-WG

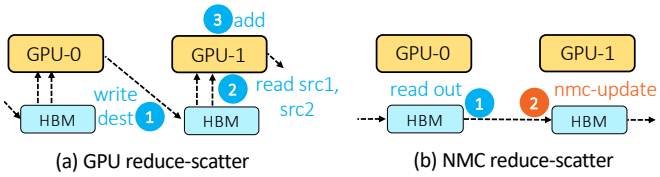


Figure 10. HBM reads & writes in steady-state reduce-scatter step.

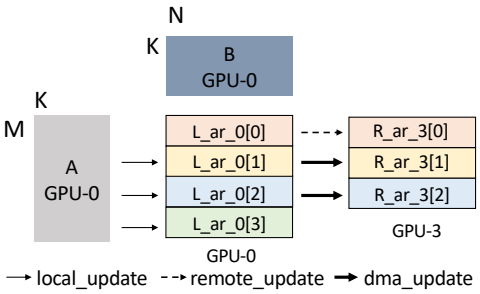


Figure 11. Remote address mapping for T3 GEMM-RS over four GPUs.

mapping amongst producer kernels. Overall, configuring the output address space mitigates the need to change GEMM implementations to enable fusion with collectives.

4.5 Communication-aware MC Arbitration (MCA):

Finally, careful scheduling of memory accesses by the producer kernel and those resulting from communication is crucial to efficiently overlap them. In Section 6.1 we show that a memory controller (MC) arbitration policy which a) round-robins between issuing memory accesses from the compute and communication streams and b) falls back to the other stream if the current stream is empty, results in producer kernel slowdowns. Communication-related memory accesses appear in bursts and can occupy DRAM queues, stalling the compute kernel's critical memory reads/writes. Simply prioritizing producer kernel accesses as they appear is also insufficient as prior communication-related memory accesses may already occupy DRAM queues. Finally, giving the local compute stream dedicated access results in wasted cycles and memory bandwidth underutilization. Thus, an efficient overlap of compute and communication requires a dynamic arbitration policy that addresses both contention and under-utilization.

We implement a simple yet dynamic arbitration policy to overcome this. The MC always prioritizes compute stream accesses, but if empty, falls back to communication stream. Additionally, it monitors the DRAM queue occupancy and only issues communication-related accesses if occupancy is below a threshold. This ensures sufficient room in the queues for future compute stream accesses and prevents their stalls. The occupancy threshold depends on the memory-intensiveness of compute kernels (e.g., smaller if memory-intensive, and vice-versa). This is determined dynamically: MC detects the memory intensiveness of a kernel by monitoring occupancy

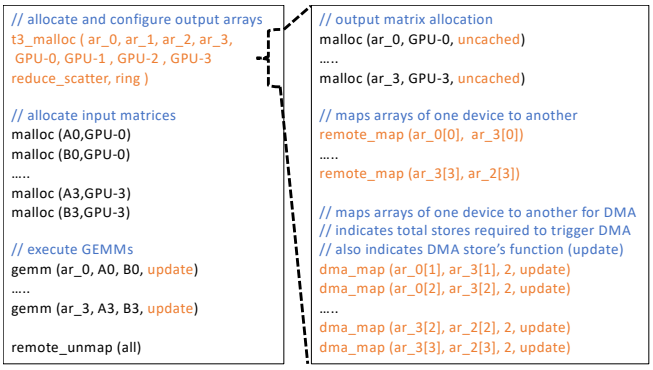


Figure 12. Configuring producer output for T3 GEMM-RS over four GPUs.

during its isolated execution (the first stage in Figure 7). Finally, the MC tracks cycles elapsed since the last issue from the communication stream and prioritizes it if it exceeds a limit to ensure it is not starved. Additionally, the communication stream is drained at the producer kernel boundary.

5 Methodology

5.1 Setup

5.1.1 Multi-GPU Simulation. Although a number of popular GPU simulators are publicly available [7, 20, 39, 74], we chose to evaluate T3 using Accel-Sim [33] because it provides high fidelity for modern GPUs [31]. Like prior work [32], we extended Accel-Sim to simulate a multi-GPU system. We observe that in a multi-GPU DNN setup all GPU's executions are homogeneous (Figures 2 and 10). Thus, we evaluate both our multi-GPU baseline and T3 by modeling all the activities pertaining to a single GPU. This includes modeling the Tracker which is accessed/updated in parallel with the store/DMA operations and uncached NMC updates. Although we do not model the DMA engine in the simulator, we do model its inter-GPU communication (communication resulting from RS both in the baseline and T3's fused GEMM-RS) by executing the compute operations (e.g., GEMM [30, 61]) in Accel-Sim and using Accel-Sim's front-end tracing functionality to inject the additional inter-GPU communication traffic. The Tracker's DMA triggering overheads are negligible since the DMA commands are pre-queued during the setup process (Figure 12) as is often done, especially for ML operations which are repetitive [24]. Table 1 details the GPU configuration we use to evaluate T3, which is the latest GPU architecture Accel-Sim completely supports. Commercial GPUs with such a configuration support a 150 GB/s interconnection ring bandwidth [53]. Since recent GPUs frequently scale compute faster than other resources, we also evaluate another configuration with increased CU count while the other parameters stay the same in Section 7.5.

Figure 13 describes our multi-GPU simulation of RS. In each RS step, a GPU performs a reduction of a sub-array and

System	
#GPUs	8, 16
Inter-GPU Interconnect	Ring, 150 GB/s bi-directional 500 ns link latency
Per-GPU Config	
#CUs	80, 1.4 GHz
Per-CU Config	2K threads, 128KB unified LDS + L1 cache (with no write-allocate), 256KB RF
L2	16MB, 64 banks, 1.4 GHz
HBM2	1 TB/s, 1 GHz, CCDWL=4, Bank Grp.=4, rest [11]

Table 1. Simulation setup.

sends it to the neighbor GPU while also receiving a reduced sub-array (corresponding to a different chunk) from another neighbor GPU (Figures 3 and 10(a)). The simulator executes the reduction of the array as-is. Simulating the incoming network traffic requires: (a) determining packet addresses, (b) generating packets at the appropriate rate, and (c) modeling the interconnect costs. Packet addresses are determined using the store trace of WGs from the reduction kernel. Next, since GPU executions are homogeneous, remote traffic is generated at the same rate as the GPU generates the reduction output (which is filtered out to be sent to remote GPU). This also implicitly includes slowdowns due to compute/-communication interference at the remote GPU. Finally, we add the interconnect costs to these packets as they arrive, assuming a simple link bandwidth and latency model of the interconnect. To validate this setup, we compare simulated RS times on four GPUs with hardware measurements from a four-GPU node with AMD Instinct™ MI210 GPUs [5] with same ring network bandwidth as simulated (Table 1). Figure 14 shows that simulation closely follows hardware trends for a range of sizes (6-192 MB): 6% geomean error versus the ideal dotted line.

Near-Memory Computing: We modify the simulator's HBM to model NMC updates. Further, memory vendor proposals indicate that NMC operations can be issued without a significant increase in DRAM timings; back-to-back NMC operations can be issued to the same bank group with the same column-to-column access (CCDL) delay [34]. To model the additional cost of NMC op-and-store operations (Section 4.3), we modify the simulator's HBM to use a 2× higher CCDL delay (termed CCDWL) following those operations (see Table 1).

5.1.2 End-to-End Transformer Iteration.

To evaluate end-to-end iterations with T3, we scale the GEMMs and RS times in the baseline Transformer breakdown (shown in Figure 4) by their simulated speedups (described in Section 5.1.1). We leverage a combination of hardware data and analytical modeling as done by prior works [49, 64] to get the end-to-end breakdowns of models in their distributed setups. We use a single-GPU mixed-precision [45] execution of MLPerf v1.1 [48] BERT on an AMD Instinct™ MI210 accelerator (GPU) [5] and scale its operation times based on

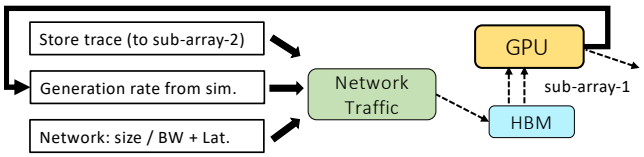


Figure 13. Simulating multi-GPU reduce-scatter.

changing hyperparameters and setup (e.g., sliced GEMM). This is beneficial as it helps us evaluate larger futuristic models (Transformer models are similar differing only in layers size/counts [49, 63]) and takes into account several GPU optimizations for Transformers [13, 15] already in MLPerf implementations. Our projections further match those measured by prior works. For example, AR's percentage runtime contribution projected for Mega-GPT-2 with TP-16 matches prior works' measurements on a similar system configuration [36].

5.2 Applications, Deployment & GEMMs

Models and their deployment: Since Transformers are fast-evolving, we evaluate T3's impact on a range of Transformer models and TP degrees (Table 2). For Megatron-GPT-2 (Mega-GPT-2) [78] and T-NLG [47] we use 16K and 8K input tokens (= input-length * batch-size) and TP degrees of eight and 16, given their modern intra-node setups [25, 36, 47, 78]. For larger Transformers like PALM [12], GPT-3 [9], and MT-NLG [80]) we use a higher slicing degree of 32 given their increasingly large memory capacity requirements [64] and availability of nodes with larger device counts that can enable this slicing [29, 59, 76]. We evaluate mixed-precision training which entails half-precision (FP16) forward and backpropagation and single-precision (FP32) weight updates. Similarly, we evaluate FP16 inference.

GEMMs: GEMMs from the aforementioned applications are simulated using implementations from state-of-the-art BLAS libraries [30, 61]. Most GEMMs (including all GEMMs we evaluate) use a tiled GEMM implementation where each WG generates a complete tile of data (other implementations discussed in Section 7.7). Further, we evaluate GEMMs with both non-transposed (e.g., backward GEMMs) and transposed (e.g., forward GEMMs) input tensors, as observed in MLPerf's BERT [44, 72].

5.3 Configurations

To evaluate T3's efficacy we use the following configurations:

- ***Sequential***: is the baseline configuration. Like modern systems, sequential executes sliced GEMMs and the following AR kernels sequentially.
- ***T3***: is our proposal which fuses and overlaps GEMM with RS (as described in Section 4), followed by sequential all-gather (AG).

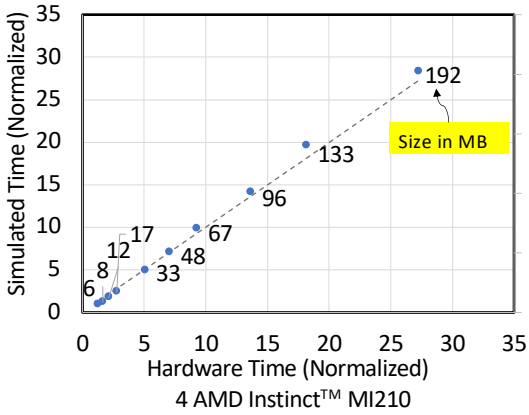


Figure 14. Validation of multi-GPU reduce-scatter simulation.

- ***T3-MCA***: uses fused GEMM-RS as in T3, but also includes the memory controller arbitration (MCA) discussed in Section 4.5.
- ***Ideal-GEMM-RS-Overlap***: represents ideal GEMM and RS overlap in software. Thus, its performance is the maximum of the GEMM's and the RS's isolated kernel execution times, followed by the AG time. Moreover, it assumes no dependency constraints or resource contention between GEMM and RS.
- ***Ideal-RS+NMC***: uses RS with near-memory computing, which can provide additional speedup beyond a perfect overlap. Thus, its performance is max(GEMM, RS+NMC) over Ideal-GEMM-RS-Overlap.

6 Results

6.1 Execution Time Distribution & Speedups

Figures 15 and 16 show results for all sliced sub-layers in Transformers which require an AR: output projection (OP) and fully-connected-2 (FC-2) in forward pass (fwd) and fully-connected-1 (FC-1) and input projection (IP) in backprop (bwd). We show these for Mega-GPT-2 and T-NLG, as well as two TP setups (TP of 8 and 16). Figure 15 shows each case's runtime distribution between the GEMM, RS, and AG. Figure 16 shows their speedup over *sequential* using *T3*, *T3-MCA*, as well as their speedups assuming an ideal overlap of GEMM with RS (*Ideal-GEMM-RS-Overlap*) and additional speedups resulting from a faster RS with NMC (*Ideal RS+NMC*).

6.1.1 Ideal Speedups. Figure 16 shows the ideal possible speedups and breaks them into two parts: first from overlapping the GEMM and RS kernels (*Ideal-GEMM-RS-Overlap*) and second from improved RS performance due to NMC (*Ideal RS+NMC*).

In Figure 16 Ideal-GEMM-RS-Overlap (without resource and data-dependency constraints) shows considerable benefits from overlapping the producer GEMM and following RS: 50% max speedup and 35% geomean versus Sequential.

Model Name	Hyperparams	Inputs	TP degree
Mega-GPT-2	H=3072, L=74	SL=1K, B=16	8, 16
T-NLG	H=4256, L=78	SL=1K, B=8	8, 16
GPT-3	H=12K, L=96	SL=1K, B=2	32
PALM	H=18K, L=118	SL=1K, B=2	32
MT-NLG	H=20K, L=105	SL=1K, B=2	32

Table 2. Studied models, their hyperparameters & setup.

Speedups vary both within and across models and depend on the isolated execution times of GEMM and RS (Figure 15). The situations where the GEMM and RS runtimes are similar (similar proportions in Figure 15) have the maximum potential since the GEMM hides all of RS's cost. For example, FC-1 in T-NLG with TP=16 obtains 50% speedup. Alternatively, the cases in which the GEMM and RS times are skewed show the least benefit since most of the GEMM or RS cost is exposed. For example, Ideal-GEMM-RS-Overlap speedup is only 15% for OP in Mega-GPT with TP=16. However, the latter is uncommon and is a consequence of slicing a very small layer (OP is the smallest among all). It does not hold for other sub-layers within the same model, or larger models as shown in the figures (also see Section 6.4). For a given hardware setup, these execution time ratios, and thus Ideal-GEMM-RS-Overlap speedups are dictated by layer parameters [64].

In Figure 16 Ideal-RS+NMC shows that additional speedup is possible beyond what perfect overlap provides. Besides freeing all the CUs for GEMMs, performing RS reductions near memory also lowers RS's memory traffic (described in Section 4.3). This speeds up RS by 7% and 3% with TP=8 and TP=16, respectively. NMC only reduces RS's final step time as interconnect costs dominate all prior steps and thus its runtime benefit decreases as TP, and thus total steps, increases. As shown in Figure 16, this faster RS can reduce overlapped time and provide additional speedups of up to 4%. Intuitively, the impact of a faster RS is only evident in layers in which RS is longer running than GEMM and is otherwise hidden when overlapped.

6.1.2 T3 Speedups. T3 transparently overlaps GEMMs with their corresponding consumer RS in a fine-grained manner. Moreover, T3's lightweight track-&-trigger mechanism and use of near-memory compute frees all CUs for GEMMs and reduces DRAM traffic (Figure 18 and Section 6.2), respectively. Thus, T3 achieves speedups of up to 39% (20% geomean, yellow bars, Figure 16).

Individual speedups vary considerably and are largely impacted by the extent of contention between DRAM traffic from the GEMM and the concurrent, RS (details in Section 6.2). For OP layers, T3 achieves close to the Ideal-GEMM-RS-Overlap speedups, and even exceeds them in certain cases. This happens because the OP GEMMs are small and fit largely in the LLC, having very small DRAM read traffic in Sequential (shown in Figure 18). Thus, the additional DRAM traffic from the overlapped RS in T3 has little impact on the GEMMs' progress/execution. Instead, T3 further improves

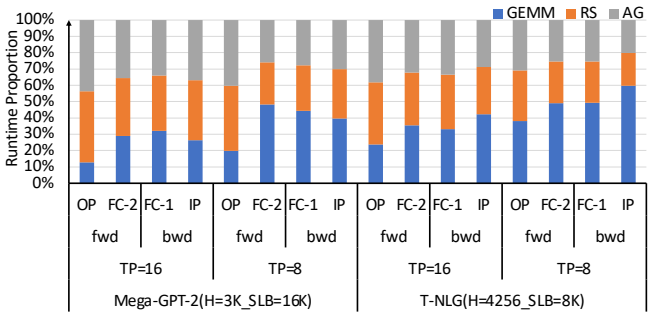


Figure 15. Transformer sub-layer runtime distribution.

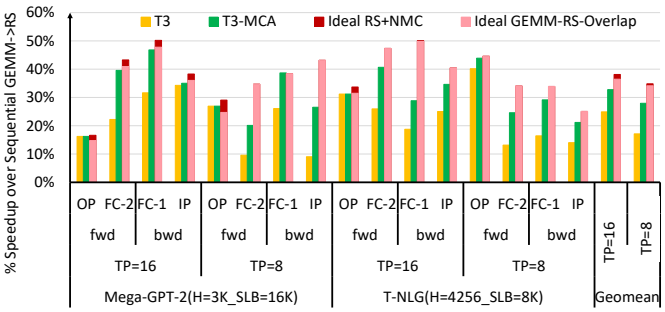


Figure 16. Transformer sub-layer speedups with T3

.

RS runtimes in these cases via NMC and enables part of the additional Ideal-RS+NMC speedups. Finally, although the track & trigger mechanism operates at a small WF granularity, generally data from multiple WFs/WGs of a GEMM stage are ready to be sent concurrently, resulting in high network bandwidth utilization. Furthermore, even when this is not true, T3 can tolerate this because compute/GEMM execution and communication are overlapped, hiding the latency.

In many other cases, and especially the much larger FC layers, the benefits are far from those with Ideal-GEMM-RS-Overlap (>15% slower). Figure 17 shows the DRAM traffic (Y-axis) and the GEMM slowdown (X-axis) with fine-grained overlapping, compared to the GEMM's isolated execution. An isolated GEMM as shown in Figure 17(a) executes in multiple stages (Section 2.5), each with a read phase (blue) followed by a bursty write phase, which limit read traffic. Overlapping RS induces additional DRAM traffic, as shown in Figure 17(b). Besides additional traffic, in T3, GEMM, and RS writes directly update memory using NMC (Section 4.3). These additional bursts of reads (RS_reads for a stage are issued as soon as both the local and neighbors' copies have updated the memory) and updates (RS_updates for the next stage from the previous neighbor) can further stall local GEMM reads as shown, causing GEMM to slow down considerably.

6.1.3 T3-MCA Speedups. T3-MCA (Section 4.5) limits GEMM reads stalls due to bursty RS traffic (Section 6.1.2, Figure 17) using a simple arbitration logic. It prevents RS traffic from completely occupying DRAM queues by limiting communication-related accesses when a DRAM queue occupancy reaches a threshold (5, 10, 30, or no limit) determined by the memory intensity of the GEMM kernel. T3-MCA provides considerable benefits over sequential execution; maximum of 47% and geomean of 30% (29% maximum and 13% geomean over T3). Furthermore, the geomean speedup with T3-MCA is only 5% smaller than Ideal-GEMM-RS-Overlap. There are individual cases where T3-MCA is far from ideal (e.g., FC-1 in T-NLG with TP=16). These represent cases where L2 bypassing (for near-memory update) of GEMM writes hurts the GEMM's performance. Consequently, the overall overlapped runtime also increases.

6.2 Data Movement Reductions

Besides improved performance, T3 and T3-MCA also reduce data movement to and from DRAM by a maximum of 36% and an average of 22% for the sub-layers. Figure 18 shows the total memory accesses and their detailed breakdown (amongst GEMM, RS and AG reads/writes) for a single GPU across all cases. While the AG reads/write remain constant between baseline (sequential) and T3-MCA, there is a combination of reasons which impact the rest: (a) fusion of GEMM and RS eliminates local writes from GEMM's first stage and reads from RS's first step, (b) near-memory reductions eliminate reading of partial copies every RS step, as well as the reads and writes in the final step's reduction, and (c) LLC bypassing of GEMM's output writes improves input read caching for cache-sensitive GEMMs, reducing GEMM's local reads. These impacts also vary depending on the TP degree: the one-time reductions (in the first and last RS step) have a much higher impact with smaller TP degrees due to fewer overall RS steps. Conversely, GEMM read caching impact is higher with a larger TP degree; larger TP/slicing leads to smaller, more LLC-amenable GEMMs. Overall, RS's reads reduce by 2.4× geomean (2.5× for TP=8, 2.2× for TP=16), both GEMM's and RS's writes reduce by 10% geomean (14% for TP=8, 7% for TP=16), and finally GEMM's reads decrease by 1.56× geomean (1.2× for TP=8, 2× for TP=16).

6.3 End-to-end Model Speedups

As shown in Figure 19, T3 and T3-MCA speed up model training by a maximum of 9% and 12%, and geomean of 7% and 10%, respectively. Benefits are higher at larger TPs due to the overall higher proportion of the sliced sub-layers requiring AR (Section 4). Similarly, prompt processing and/or large input token processing during inference is also sped up by a maximum of 12% and 15%, and geomean of 9% and 12% with T3 and T3-MCA, respectively. Inference speedups are better due to the overall higher proportion of sliced sub-layers resulting from no backprop compute. Finally, the MLPerfv1.1 implementation we evaluate does not include a key fusion optimization [13], which makes the non-sliced attention operations a significant 40-45% of execution time. Thus, we

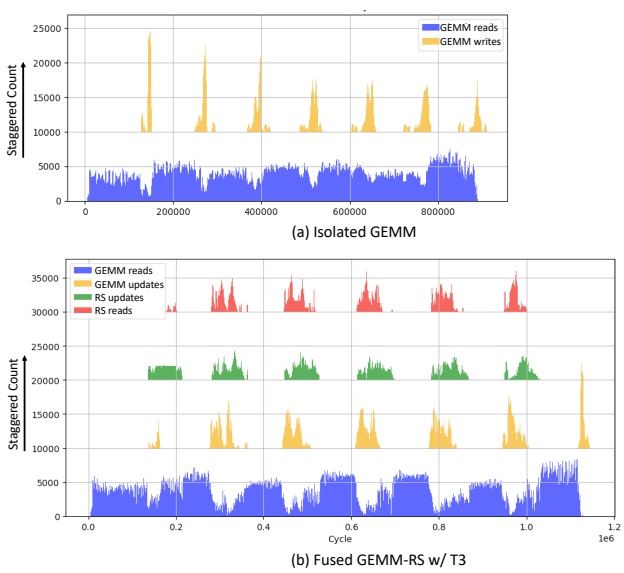


Figure 17. Overall DRAM traffic in (a) baseline GEMM, (b) T3, for T-NLG FC-2 with TP=8 and SLB=4K.

expect T3's and T3-MCA's benefits to be much higher for newer MLPerf implementations.

6.4 Impact on Larger Transformers

We also evaluate larger Transformers with higher TP degrees as shown in Figure 19. Similar to the smaller models, layer-level speedups are high; max 35% and geomean of 29% for GPT-3 (175B parameters), PALM (530B parameters), and MT-NLG (540B parameters). These lead to up to 12% and 14% end-to-end speedup in their training and prompt phase of inference, respectively. Thus, T3-MCA also effectively speeds up larger models.

7 Discussion

7.1 Other Collectives Implementation & Types

T3 supports other collectives and implementations via the configuration of GEMM's output address space (Section 4.4).

Other implementations: Collectives can have multiple implementations optimized for different topologies. We focus on ring since it is commonly used in intra-node setups where tensor slicing is employed [26]. T3 can also support the *direct* RS implementation in a fully-connected topology. At every GEMM stage, the output from each device is scattered across the remaining devices using dedicated links and reduced at the destination. This is accomplished by changing the configuration in Figure 12 to slice each GEMM stage output and remote_map each slice to a remote device. In this case T3 eliminates memory accesses by the collective as it is completely orchestrated using GEMM stores. Similarly, it can also support other, inter-node implementations via appropriate programming of the track & trigger mechanism.

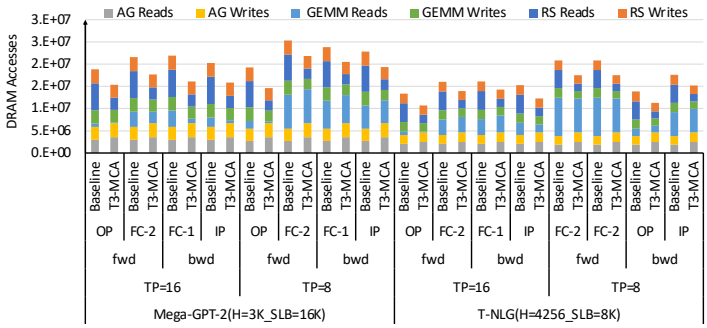


Figure 18. DRAM access per sub-layer.

Other types: Similarly, T3 also supports other collectives. A ring/direct all-gather (AG) reuses ring-RS's configuration and setup, except the GEMMs and DMA transfers do not update memory locations. Similar to AG, T3 can also support an all-to-all collective where devices exchange sub-arrays, except here the remote/dma_mapped GEMM output is not written to local memory.

7.2 Other Distributed Techniques

Although we focus on communication in tensor-parallel (TP) setups, T3 is also applicable in other distributed setups where a producer's output is communicated via a collective.

Expert Parallelism: Similar to TP, expert parallelism in Mixture-of-experts (MoEs) [17, 69] require serialized all-to-all communication which can be fused with T3 as discussed in Section 7.1.

Data & Pipeline Parallelism: T3 also applies to data-parallel and pipeline-parallel setups which require RS, and peer-to-peer transfers, respectively. While T3's overlapping benefits may not provide additional benefits in such cases (these communications can be overlapped with other independent kernels), T3's NMC and MCA techniques can help reduce memory bandwidth contention in these cases as well.

TP with All-gather: T3 can be extended for distributed setups where the collective's output requires overlapping with a long-running consumer operation. This is required if the producer is short-running (e.g., TP which all-gather's activations). Overlapping collective-consumer pairs is similar in principle to overlapping producer-collective and requires similar tracking/triggering mechanisms. The Tracker would track "all-gathered-input→GEMM-WG" instead of "GEMM-WG→all-reduced-output". Moreover, instead of triggering a DMA, it would trigger a WG scheduling event (such as in Lustig & Martonosi [42]). This can be challenging since the "all-gathered-input→GEMM-WG" mapping can be kernel implementation dependent. However, additional programming hints could overcome this.

7.3 Generative Inference

While we focus on the communication-heavy training and prompt phase of inference, T3 is also applicable in the generation phase of inference. Due to smaller input token counts

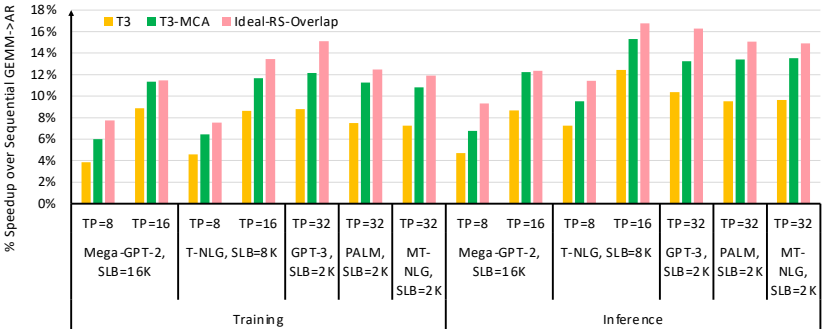


Figure 19. End-to-end model speedups.

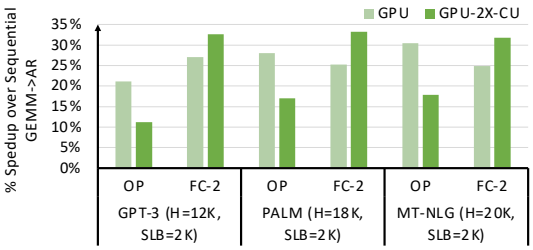


Figure 20. T3 on future hardware with 2× compute.

(Section 2.1), these phases are bound by memory accesses of model weights and can benefit from the aggregate memory bandwidth of multiple devices that TP provides [6]. The resulting all-reduce of activations, while smaller than those in training and thus potentially latency-bound (due to small token counts), can still be overlapped and hidden with GEMM executions using T3.

7.4 Other Reduction Substrates

While T3 leverages NMC for atomic updates required in reduction-based collectives (e.g., RS, AR), it is not a requirement. Such updates could also be handled via system-wide atomics on uncached data without significant loss in performance. Similarly, T3 can also leverage switches for reductions as shown by prior works [36].

7.5 Future Hardware & Lower Precision

Since compute FLOPS have scaled much more than network link bandwidths across hardware generations [19, 54, 56, 60], communication will likely be a larger proportion of the end-to-end execution both in more recent systems than the one we evaluate and in the future. Similarly, lowering precision [46, 75] decreases compute time much more (quadratically) than communication (linearly). Thus, the benefits of hiding communication with techniques like T3 will also apply to other GPU configurations and datatypes besides 16b.

To evaluate T3's hiding capability in future systems, we study a system configuration where compute FLOPS scale more than network link bandwidth (2×), which we term GPU-2X-CU. While the scaling GPU FLOPs across generations largely result from more powerful CUs (larger/faster tensor processing), we simulate it by scaling the number of CUs and keeping the underlying network the same. This enables us to use the latest/validated GPU model and GEMM traces that Accel-Sim supports [33]. Figure 20 shows that for larger layers (FC-2) where compute time dominates, compute becomes faster with 2× CUs which lowers the compute:communication ratio across the models. This shortens the critical path and leads to larger benefits with overlapping compute and communication with T3. Conversely, for smaller layers (OP), where compute and communication are more balanced, faster compute exposes communication on critical path, lowering T3's benefits. Note that, for such scenarios, communication optimizations will be necessary [10, 77]. Nevertheless, the larger layers have a more prominent impact on overall execution and for these, T3's benefits only improve.

7.6 NMC for Following Operations

Collectives, specifically all-reduce in Transformers, are usually followed by other memory-intensive operations on all devices (e.g., parameter updates in DP [63] or residual/dropout layers in TP). These operations redundantly operate on the entire all-reduced array on each device. With T3, these following memory-intensive operations can be executed using NMC [63] on (reduced) sub-arrays of data before they are all-gathered/broadcasted to the remaining devices, thus reducing redundancy, and further accelerating distributed Transformer models.

7.7 Other GEMM Implementations

T3 focuses on the most common tiled GEMM implementation with a WG/WF responsible to generate an entire tile/sub-tile of data. However, T3 can support other implementations, such as split-K [58]. A split-K implementation slices work in the accumulation or K dimension, such that multiple WGs are responsible for a single tile, each generating a partial tile that is reduced after. Split-K increases parallelism when the output size (MxN) is small but the K dimension is large. However, tensor-sliced GEMMs, which require AR, have large output sizes and small K dimensions. Naively, T3 with a split-K implementation (with more than one update to an element) will cause multiple local and remote updates per memory location. To prevent this, T3 can use the kernel packets' tile-size metadata to deduce split-k degree (=(#WGs * tile-size)/(M*N)), i.e., the number of updates per element. The virtual addresses in the tracker (Section 4.2.1) can be used to determine WFs/WGs/tracker entries that update the same tile, allowing the tracker to trigger remote DMA only after all updates to the tile are complete.

Approach / Features	GPU Support	Transparent Communication	Overlap Communication	Reduce Contention	No Additional Accelerator	Topology-independent
In-switch [36]	✓	X	X	✓	X	X
ACE [71]	✓	X	X	✓	X	X
CoCoNet [25]	✓	X	✓	X	✓	✓
Google Decomposition [86]	X	X	✓	X	✓	✓
T3-MCA	✓	✓	✓	✓	✓	✓

Table 3. Comparing T3-MCA to prior work.

7.8 Multi-node Setups

Tensor-parallelism, with serialized communication is usually employed within a node, which generally has high-speed homogeneous links. However, T3 can also be applied to serialized communication in inter-node setups with slower and often heterogeneous links. Consequently, communication costs can be much larger than GEMM executions, potentially limiting the benefits from fine-grained overlap: once the computation is completely overlapped, the remaining communication costs will be exposed [86]. Nevertheless, T3 can still provide benefits from hiding the GEMM execution cost as much as possible.

8 Related Work

Table 3 compares T3-MCA with prior works across several key metrics. Some prior work has designed *in-switch collectives* to speed up communication by up to 2× [36]. However, this cannot eliminate serialized communication from the critical path. Furthermore, they are topology-dependent, requiring switches. Enabling fine-grained overlap of compute and communication is essential to effectively hide the cost of communication. Existing attempts to do this, like *CocoNet*[25] and *Google Decomposition* [86], have limitations. *Google Decomposition* requires changes to matrix multiplication (GEMMs) kernels which can be disruptive to GPU software infrastructure (Section 3.1).

Furthermore, both approaches can suffer from hardware resource contention between compute and communication (Section 3.2). Works that reduce contention only address coarse-grained overlap of compute and communication in cases like DP, lacking support for fine-grained overlap in serialized collectives [71]. Moreover, they rely on dedicated accelerators. Other recent work fuses communication within the computation kernel to enable fine-grained overlap, such that a GPU kernel performs both computation and dependent communication at the WG level [67]. However, this requires explicit changes to the compute kernels and is not readily applicable for collectives involving simple arithmetic operation such as reduce-scatter – which will still be limited by inter-GPU synchronization. Finally, other work like Syndicate increases coarse-grained overlap opportunities and efficiency in distributed training. However, Syndicate cannot hide serialized communication [43]. *T3-MCA overcomes these shortcomings and achieves a transparent overlap of serialized communication with compute, while minimizing resource contention.*

9 Conclusion

Transformer models increasingly rely on distributed techniques, requiring communication between multiple devices. This communication can limit scaling efficiency, especially for techniques like Tensor Parallelism (TP) which serialize communication with model execution. While a fine-grained overlap of the serialized communication with its producer computation can help hide the cost, realizing it with GPUs is challenging due to software complexities and resource contention between compute and communication. To overcome this, we propose T3, which transparently and efficiently fuses and overlaps serialized inter-device communication with the producer's compute. It orchestrates communication on the producer's stores by configuring the producer's output address space mapping and using a programmable track and trigger mechanism in hardware. This reduces application impact and also eliminates contention for GPU compute resources. T3 additionally uses near-memory computing and a memory-controller arbitration policy to reduce memory-bandwidth contention. Overall, T3 improves performance by 30% geomean (max 47%) and reduces data movement by 22% geomean (max 36%) over state-of-the-art approaches. Moreover, T3's benefits hold as models and hardware scale.

Acknowledgements

We would like to thank our shepherd, Saeed Maleki, and the anonymous reviewers for their feedback that helped improve this paper. We would also like to thank Mahmoud Khairy Abdallah for his help with the Accel-Sim simulator. This work is supported in part at the University of Wisconsin-Madison by the Vilas Life Cycle Professorship program and a Fall Research Competition grant, as well as the National Science Foundation under grant ENS-1925485. AMD, AMD Ryzen, AMD Radeon, and combinations thereof are trademarks of Advanced Micro Devices, Inc. Other product names used in this publication are for identification purposes only and may be trademarks of their respective companies.

References

[1] AMD. 2018. AMD's ROCm Communication Collectives Library. "https://github.com/ROCmSoftwarePlatform/rccl/wiki".

[2] AMD. 2019. AMD's BLAS Library. "https://github.com/ROCmSoftwarePlatform/rocBLAS".

[3] AMD. 2020. AMD's tool for creating a benchmark-driven backend library for GEMMs. "https://github.com/ROCmSoftwarePlatform/Tensile/".

[4] AMD. 2021. AMD HSA Code Object Format. "https://rocmdocs.amd.com/en/latest/ROCm_Compiler_SDK/ROCm-Codeobj-format.html".

[5] AMD. 2022. AMD INSTINCT™ MI210 ACCELERATOR. https://www.amd.com/en/products/server-accelerators/amd-instinct-mi210.

[6] Reza Yazdani Aminabadi, Samyam Rajbhandari, Ammar Ahmad Awan, Cheng Li, Du Li, Elton Zheng, Olatunji Ruwase, Shaden Smith, Minjia Zhang, Jeff Rasley, and Yuxiong He. 2022. DeepSpeed-Inference: Enabling Efficient Inference of Transformer Models at Unprecedented Scale. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*. IEEE, IEEE Press, Piscataway, NJ, USA, 1–15.

[7] Yuhui Bao, Yifan Sun, Zlatan Feric, Michael Tian Shen, Micah Weston, José L. Abellán, Trinayan Baruah, John Kim, Ajay Joshi, and David Kaeli. 2023. NaviSim: A Highly Accurate GPU Simulator for AMD RDNA GPUs. In *Proceedings of the International Conference on Parallel Architectures and Compilation Techniques* (Chicago, Illinois) *(PACT '22)*. Association for Computing Machinery, New York, NY, USA, 333–345. https://doi.org/10.1145/3559009.3569666

[8] Nathan Benaich and Ian Hogarth. 2022. State of AI Report 2022. https://www.stateof.ai/.

[9] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language Models are Few-Shot Learners. In *Advances in Neural Information Processing Systems (NeurIPS, Vol. 33)*, H. Larochelle, M. Ranzato, R. Hadsell, M. F. Balcan, and H. Lin (Eds.). Curran Associates Inc., Red Hook, NY, USA, 1877–1901.

[10] Zixian Cai, Zhengyang Liu, Saeed Maleki, Madanlal Musuvathi, Todd Mytkowicz, Jacob Nelson, and Olli Saarikivi. 2021. Synthesizing Optimal Collective Algorithms. In *Proceedings of the 26th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPOPP)*. Association for Computing Machinery, New York, NY, USA, 62–75. https://doi.org/10.1145/3437801.3441620

[11] Niladrish Chatterjee, Mike O'Connor, Donghyuk Lee, Daniel R Johnson, Stephen W Keckler, Minsoo Rhu, and William J Dally. 2017. Architecting an Energy-Efficient DRAM System for GPUs. In *23rd IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, IEEE Computer Society, Washington, DC, USA, 73–84.

[12] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, Parker Schuh, Kensen Shi, Sasha Tsvyashchenko, Joshua Maynez, Abhishek Rao, Parker Barnes, Yi Tay, Noam Shazeer, Vinodkumar Prabhakaran, Emily Reif, Nan Du, Ben Hutchinson, Reiner Pope, James Bradbury, Jacob Austin, Michael Isard, Guy Gur-Ari, Pengcheng Yin, Toju Duke, Anselm Levskaya, Sanjay Ghemawat, Sunipa Dev, Henryk Michalewski, Xavier Garcia, Vedant Misra, Kevin Robinson, Liam Fedus, Denny Zhou, Daphne Ippolito, David Luan, Hyeontaek Lim, Barret Zoph, Alexander Spiridonov, Ryan Sepassi, David Dohan, Shivani Agrawal, Mark Omernick, Andrew M. Dai, Thanumalayan Sankaranarayana Pillai, Marie Pellat, Aitor Lewkowycz, Erica Moreira, Rewon Child, Oleksandr Polozov, Katherine Lee, Zongwei Zhou, Xuezhi Wang, Brennan Saeta, Mark Diaz, Orhan Firat, Michele Catasta, Jason Wei, Kathy Meier-Hellstern, Douglas Eck, Jeff Dean, Slav Petrov, and Noah Fiedel. 2022. PaLM: Scaling Language Modeling with Pathways. *arXiv preprint arXiv:2204.02311* (2022), 87 pages.

[13] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. FlashAttention: Fast and Memory-efficient Exact Attention with IO-Awareness. *Advances in Neural Information Processing Systems* 35 (2022), 16344–16359.

[14] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, Jill Burstein, Christy Doran, and Thamar Solorio (Eds.). Association for Computational Linguistics, Morristown, NJ, USA, 4171–4186. https://doi.org/10.18653/v1/n19-1423

[15] Shraf Eassa and Sukru Burc Eryilmaz. 2022. The Full Stack Optimization Powering NVIDIA MLPerf Training v2.0 Performance. https://developer.nvidia.com/blog/boosting-mlperf-training-performance-with-full-stack-optimization/.

[16] Izzat El Hajj, Juan Gomez-Luna, Cheng Li, Li-Wen Chang, Dejan Milojicic, and Wen-mei Hwu. 2016. KLAP: Kernel Launch Aggregation and Promotion for Optimizing Dynamic Parallelism. In *49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, IEEE Press, Piscataway, NJ, USA, 1–12. https://doi.org/10.1109/MICRO.2016.7783716

[17] William Fedus, Barret Zoph, and Noam Shazeer. 2022. Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity. *The Journal of Machine Learning Research* 23, 1, Article 120 (jan 2022), 39 pages.

[18] Jan Fousek, Jiři Filipovič, and Matuš Madzin. 2011. Automatic Fusions of CUDA-GPU Kernels for Parallel Map. *SIGARCH Comput. Archit. News* 39, 4 (Dec. 2011), 98–99. https://doi.org/10.1145/2082156.2082183

[19] Amir Gholami. 2021. AI and Memory Wall.

[20] Anthony Gutierrez, Bradford M. Beckmann, Alexandru Dutu, Joseph Gross, Michael LeBeane, John Kalamatianos, Onur Kayiran, Matthew Poremba, Brandon Potter, Sooraj Puthoor, Matthew D. Sinclair, Michael Wyse, Jieming Yin, Xianwei Zhang, Akshay Jain, and Timothy Rogers. 2018. Lost in Abstraction: Pitfalls of Analyzing GPUs at the Intermediate Language Level. In *24th IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE Computer Society, Los Alamitos, CA, USA, 608–619. https://doi.org/10.1109/HPCA.2018.00058

[21] Hany Hassan Awadalla, Anthony Aue, Chang Chen, Vishal Chowdhary, Jonathan Clark, Christian Federmann, Xuedong Huang, Marcin Junczys-Dowmunt, Will Lewis, Mu Li, Shujie Liu, Tie-Yan Liu, Renqian Luo, Arul Menezes, Tao Qin, Frank Seide, Xu Tan, Fei Tian, Lijun Wu, Shuangzhi Wu, Yingce Xia, Dongdong Zhang, Zhirui Zhang, and Ming Zhou. 2018. Achieving Human Parity on Automatic Chinese to English News Translation. *arXiv preprint arXiv:1803.05567* (March 2018), 25 pages. arXiv:1803.05567 [cs.CL]

[22] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2015. Deep Residual Learning for Image Recognition. *CoRR* abs/1512.03385 (2015), 12 pages. arXiv:1512.03385 http://arxiv.org/abs/1512.03385

[23] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Mia Xu Chen, Dehao Chen, HyoukJoong Lee, Jiquan Ngiam, Quoc V. Le, Yonghui Wu, and Zhifeng Chen. 2019. GPipe: Efficient Training of Giant Neural Networks using Pipeline Parallelism. In *Proceedings of the 33rd International Conference on Neural Information Processing Systems (NeurIPS, Vol. 32)*. Curran Associates Inc., Red Hook, NY, USA, Article 10, 10 pages.

[24] Changho Hwang, KyoungSoo Park, Ran Shu, Xinyuan Qu, Peng Cheng, and Yongqiang Xiong. 2023. ARK: GPU-driven Code Execution for Distributed Deep Learning. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. USENIX Association, Boston, MA, 87–101. https://www.usenix.org/conference/nsdi23/presentation/hwang

[25] Abhinav Jangda, Jun Huang, Guodong Liu, Amir Hossein Nodehi Sabet, Saeed Maleki, Youshan Miao, Madanlal Musuvathi, Todd Mytkowicz, and Olli Saarikivi. 2022. Breaking the Computation and Communication Abstraction Barrier in Distributed Machine Learning Workloads. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. Association for Computing Machinery, New York, NY, USA, 402–416. https://doi.org/10.1145/3503222.3507778

[26] Sylvain Jeaugey. 2022. How is tree reduction implemented? https://github.com/NVIDIA/nccl/issues/545#issuecomment-1006361565.

[27] Adwait Jog, Evgeny Bolotin, Zvika Guz, Mike Parker, Stephen W Keckler, Mahmut T Kandemir, and Chita R Das. 2014. Application-aware Memory System for Fair and Efficient Execution of Concurrent GPGPU Applications. In *Proceedings of Workshop on General Purpose Processing using GPUs (GPGPU)*. Association for Computing Machinery, New York, NY, USA, 1–8. https://doi.org/10.1145/2588768.2576780

[28] Adwait Jog, Onur Kayiran, Ashutosh Pattnaik, Mahmut T Kandemir, Onur Mutlu, Ravishankar Iyer, and Chita R Das. 2016. Exploiting Core Criticality for Enhanced GPU Performance. In *Proceedings of the 2016 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Science*. Association for Computing Machinery, New York, NY, USA, 351–363. https://doi.org/10.1145/2896377.2901468

[29] Norman P. Jouppi, Doe Hyun Yoon, Matthew Ashcraft, Mark Gottscho, Thomas B. Jablin, George Kurian, James Laudon, Sheng Li, Peter Ma, Xiaoyu Ma, Nishant Patil, Sushma Prasad, Clifford Young, Zongwei Zhou, and David Patterson. 2021. Ten Lessons from Three Generations Shaped Google's TPUv4i. In *Proceedings of the 48th Annual International Symposium on Computer Architecture* (Virtual Event, Spain) *(ISCA)*. IEEE Press, Piscataway, NJ, USA, 1–14. https://doi.org/10.1109/ISCA52012.2021.00010

[30] Andrew Kerr, Duane Merrill, Julien Demouth, and John Tran. 2017. cuTLASS: Fast linear algebra in CUDA C++.

[31] Mahmoud Khairy, Akshay Jain, Tor M. Aamodt, and Timothy G. Rogers. 2018. Exploring Modern GPU Memory System Design Challenges through Accurate Modeling. *CoRR* abs/1810.07269 (2018), 10 pages. arXiv:1810.07269 http://arxiv.org/abs/1810.07269

[32] Mahmoud Khairy, Vadim Nikiforov, David Nellans, and Timothy G. Rogers. 2020. Locality-Centric Data and Threadblock Management for Massive GPUs. In *53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE Computer Society, Los Alamitos, CA, USA, 1022–1036. https://doi.org/10.1109/MICRO50266.2020.00086

[33] Mahmoud Khairy, Zhesheng Shen, Tor M. Aamodt, and Timothy G. Rogers. 2020. Accel-Sim: An Extensible Simulation Framework for Validated GPU Modeling. In *ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. IEEE Press, Piscataway, NJ, USA, 473–486. https://doi.org/10.1109/ISCA45697.2020.00047

[34] Heesu Kim, Hanmin Park, Taehyun Kim, Kwanheum cho, Eojin Lee, Soojung Ryu, Hyuk-Jae Lee, Kiyoung Choi, and Jinho Lee. 2021. GradPIM: A Practical Processing-in-DRAM Architecture for Gradient Descent. In *27th IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE Computer Society, Washington, DC, USA, 14 pages.

[35] Young Jin Kim, Ammar Ahmad Awan, Alexandre Muzio, Andres Felipe Cruz Salinas, Liyang Lu, Amr Hendy, Samyam Rajbhandari, Yuxiong He, and Hany Hassan Awadalla. 2021. Scalable and Efficient MoE Training for Multitask Multilingual Models. https://doi.org/10.48550/ARXIV.2109.10465

[36] Benjamin Klenk, Nan Jiang, Greg Thorson, and Larry Dennison. 2020. An In-Network Architecture for Accelerating Shared-Memory Multiprocessor Collectives. In *ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, IEEE Computer Society, Washington, DC, USA, 996–1009.

[37] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. 2012. ImageNet Classification with Deep Convolutional Neural Networks. In *Proceedings of the 25th International Conference on Neural Information Processing Systems - Volume 1* (Lake Tahoe, Nevada) *(NIPS'12)*. Curran Associates Inc., USA, 1097–1105. http://dl.acm.org/citation.cfm?id=2999134.2999257

[38] Sukhan Lee, Shin-haeng Kang, Jaehoon Lee, Hyeonsu Kim, Eojin Lee, Seungwoo Seo, Hosang Yoon, Seungwon Lee, Kyounghwan Lim, Hyunsung Shin, Jinhyun Kim, O Seongil, Anand Iyer, David Wang, Kyomin Sohn, and Nam Sung Kim. 2021. Hardware Architecture and Software Stack for PIM Based on Commercial DRAM Technology: Industrial Product. In *ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. IEEE Press, Piscataway, NJ, USA, 43–56. https://doi.org/10.1109/ISCA52012.2021.00013

[39] Jonathan Lew, Deval A Shah, Suchita Pati, Shaylin Cattell, Mengchi Zhang, Amruth Sandhupatla, Christopher Ng, Negar Goli, Matthew D Sinclair, Timothy G Rogers, and Tor Aamodt. 2019. Analyzing Machine Learning Workloads Using a Detailed GPU Simulator. In *IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. IEEE, IEEE Computer Society, Washington, DC, USA, 151–152.

[40] Min Lin, Qiang Chen, and Shuicheng Yan. 2014. Network In Network. In *2nd International Conference on Learning Representations (ICLR)*, Yoshua Bengio and Yann LeCun (Eds.). OpenReview.net, 10 pages. http://arxiv.org/abs/1312.4400

[41] Shih-Chieh Lin, Yunqi Zhang, Chang-Hong Hsu, Matt Skach, Md E. Haque, Lingjia Tang, and Jason Mars. 2018. The Architectural Implications of Autonomous Driving: Constraints and Acceleration. In *Proceedings of the Twenty-Third International Conference on Architectural Support for Programming Languages and Operating Systems* (Williamsburg, VA, USA) *(ASPLOS)*. ACM, New York, NY, USA, 751–766. https://doi.org/10.1145/3173162.3173191

[42] Daniel Lustig and Margaret Martonosi. 2013. Reducing GPU Offload Latency via Fine-Grained CPU-GPU Synchronization. In *Proceedings of the 19th International Symposium on High Performance Computer Architecture (HPCA)*. IEEE Computer Society, USA, 354–365. https://doi.org/10.1109/HPCA.2013.6522332

[43] Kshiteej Mahajan, Ching-Hsiang Chu, Srinivas Sridharan, and Aditya Akella. 2023. Better Together: Jointly Optimizing ML Collective Scheduling and Execution Planning using SYNDICATE. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. USENIX Association, Boston, MA, 809–824. https://www.usenix.org/conference/nsdi23/presentation/mahajan

[44] Peter Mattson, Christine Cheng, Cody Coleman, Greg Diamos, Paulius Micikevicius, David A. Patterson, Hanlin Tang, Gu-Yeon Wei, Peter Bailis, Victor Bittorf, David Brooks, Dehao Chen, Debojyoti Dutta, Udit Gupta, Kim M. Hazelwood, Andrew Hock, Xinyuan Huang, Bill Jia, Daniel Kang, David Kanter, Naveen Kumar, Jeffery Liao, Guokai Ma, Deepak Narayanan, Tayo Oguntebi, Gennady Pekhimenko, Lillian Pentecost, Vijay Janapa Reddi, Taylor Robie, Tom St. John, Carole-Jean Wu, Lingjie Xu, Cliff Young, and Matei Zaharia. 2019. MLPerf Training Benchmark. *CoRR* abs/1910.01500 (2019), 14 pages. arXiv:1910.01500 http://arxiv.org/abs/1910.01500

[45] Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, and Hao Wu. 2018. Mixed Precision Training. arXiv:1710.03740 [cs.AI] http://arxiv.org/abs/1710.03740

[46] Paulius Micikevicius, Dusan Stosic, Neil Burgess, Marius Cornea, Pradeep Dubey, Richard Grisenthwaite, Sangwon Ha, Alexander Heinecke, Patrick Judd, John Kamalu, Naveen Mellempudi, Stuart Oberman, Mohammad Shoeybi, Michael Siu, and Hao Wu. 2022. FP8 Formats for Deep Learning. *CoRR* abs/2209.05433 (2022), 9 pages. https://doi.org/10.48550/ARXIV.2209.05433 arXiv:2209.05433

[47] Microsoft. 2020. Turing-NLG: A 17-billion-parameter language model by Microsoft. *Microsoft Research Blog* 1, 8 (2020), 8 pages. https://www.microsoft.com/en-us/research/blog/turing-nlg-a-17-billion-parameter-language-model-by-microsoft/

[48] MLPerf. 2018. MLPerf Benchmark Suite. https://mlperf.org/.

[49] Diksha Moolchandani, Joyjit Kundu, Frederik Ruelens, Peter Vrancx, Timon Evenblij, and Manu Perumkunnil. 2023. AMPeD: An Analytical Model for Performance in Distributed Training of Transformers. In *IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. IEEE Computer Society, Los Alamitos, CA, USA, 306–315. https://doi.org/10.1109/ISPASS57527.2023.00037

[50] Harini Muthukrishnan, Daniel Lustig, David Nellans, and Thomas Wenisch. 2021. GPS: A Global Publish-Subscribe Model for Multi-GPU Memory Management. In *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture* (Virtual Event, Greece) *(MICRO '21)*. Association for Computing Machinery, New York, NY, USA, 46–58. https://doi.org/10.1145/3466752.3480088

[51] Harini Muthukrishnan, David Nellans, Daniel Lustig, Jeffrey A Fessler, and Thomas F Wenisch. 2021. Efficient Multi-GPU Shared Memory via Automatic Optimization of Fine-grained Rransfers. In *ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, IEEE Computer Society, Washington, DC, USA, 139–152.

[52] Lifeng Nai, Ramyad Hadidi, Jaewoong Sim, Hyojong Kim, Pranith Kumar, and Hyesoon Kim. 2017. GraphPIM: Enabling Instruction-level PIM Offloading in Graph Computing Frameworks. In *IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, IEEE Computer Society, Los Alamitos, CA, USA, 457–468. https://doi.org/10.1109/HPCA.2017.54

[53] NVIDIA. 2017. NVIDIA DGX-1 With Tesla V100 System Architecture. https://images.nvidia.com/content/pdf/dgx1-v100-system-architecture-whitepaper.pdf.

[54] NVIDIA. 2018. NVIDIA TESLA V100 GPU ACCELERATOR. https://images.nvidia.com/content/technologies/volta/pdf/tesla-volta-v100-datasheet-letter-fnl-web.pdf.

[55] NVIDIA. 2020. NVIDIA NCCL.

[56] NVIDIA. 2021. NVIDIA A100 TENSOR CORE GPU. https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/a100/pdf/nvidia-a100-datasheet-us-nvidia-1758950-r4-web.pdf.

[57] NVIDIA. 2022. GPUDirect. "https://developer.nvidia. com/gpudirect".

[58] NVIDIA. 2023. Efficient GEMM in CUDA. https://github.com/NVIDIA/cutlass/blob/main/media/docs/efficient_gemm.md#parallelized-reductions.

[59] NVIDIA. 2023. NVIDIA Announces DGX GH200 AI Supercomputer. https://nvidianews.nvidia.com/news/nvidia-grace-hopper-superchips-designed-for-accelerated-generative-ai-enter-full-production.

[60] NVIDIA. 2023. NVIDIA H100 TENSOR CORE GPU. https://resources.nvidia.com/en-us-tensor-core/nvidia-tensor-core-gpu-datasheet.

[61] NVIDIA Corp. 2016. NVIDIA cuBLAS. https://developer.nvidia.com/cublas. Accessed August 6, 2016.

[62] Pratyush Patel, Esha Choukse, Chaojie Zhang, Íñigo Goiri, Aashaka Shah, Saeed Maleki, and Ricardo Bianchini. 2023. Splitwise: Efficient Generative LLM Inference Using Phase Splitting. *arXiv preprint arXiv:2311.18677* (2023), 12 pages. arXiv:2311.18677 [cs.AR]

[63] Suchita Pati, Shaizeen Aga, Nuwan Jayasena, and Matthew D. Sinclair. 2022. Demystifying BERT: System Design Implications. In *2022 IEEE International Symposium on Workload Characterization (IISWC)*. IEEE Computer Society, Los Alamitos, CA, USA, 296–309. https://doi.org/10.1109/IISWC55918.2022.00033

[64] Suchita Pati, Shaizeen Aga, Nuwan Jayasena, and Matthew D. Sinclair. 2023. Tale of Two Cs: Computation vs. Communication Scaling for Future Transformers on Future Hardware. In *IEEE International Symposium on Workload Characterization (IISWC)*. IEEE Computer Society, Los Alamitos, CA, USA, 140–153. https://doi.org/10.1109/IISWC59245.2023.00026

[65] Ashutosh Pattnaik, Xulong Tang, Onur Kayiran, Adwait Jog, Asit Mishra, Mahmut T Kandemir, Anand Sivasubramaniam, and Chita R Das. 2019. Opportunistic Computing in GPU Architectures. In *Proceedings of the 46th International Symposium on Computer Architecture (ISCA)*. Association for Computing Machinery, New York, NY, USA, 210–223. https://doi.org/10.1145/3307650.3322212

[66] J Thomas Pawlowski. 2011. Hybrid Memory Cube (HMC). In *2011 IEEE Hot Chips 23 Symposium (HotChips)*. IEEE, IEEE, Piscataway, NJ, USA, 1–24.

[67] Kishore Punniyamurthy, Bradford M Beckmann, and Khaled Hamidouche. 2023. GPU-initiated Fine-grained Overlap of Collective Communication with Computation. *arXiv preprint arXiv:2305.06942* (2023), 13 pages. arXiv:2305.06942 [cs.DC]

[68] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. 2019. Language Models are Unsupervised Multitask Learners. *OpenAI blog* 1, 8 (2019), 9.

[69] Samyam Rajbhandari, Conglong Li, Zhewei Yao, Minjia Zhang, Reza Yazdani Aminabadi, Ammar Ahmad Awan, Jeff Rasley, and Yuxiong He. 2022. DeepSpeed-MOE: Advancing Mixture-of-Experts Inference and Training to Power Next-Generation AI Scale. In *International Conference on Machine Learning (ICML)*. PMLR, PMLR, 18332–18346.

[70] Samyam Rajbhandari, Olatunji Ruwase, Jeff Rasley, Shaden Smith, and Yuxiong He. 2021. ZeRO-Infinity: Breaking the GPU Memory Wall for Extreme Scale Deep Learning. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis* (St. Louis, Missouri) *(SC '21)*. Association for Computing Machinery, New York, NY, USA, Article 59, 14 pages. https://doi.org/10.1145/3458817.3476205

[71] Saeed Rashidi, Matthew Denton, Srinivas Sridharan, Sudarshan Srinivasan, Amoghavarsha Suresh, Jade Nie, and Tushar Krishna. 2021. Enabling Compute-Communication Overlap in Distributed Deep Learning Training Platforms. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, IEEE Press, Piscataway, NJ, USA, 540–553. https://doi.org/10.1109/ISCA52012.2021.00049

[72] V. J. Reddi, C. Cheng, D. Kanter, P. Mattson, G. Schmuelling, C. Wu, B. Anderson, M. Breughe, M. Charlebois, W. Chou, R. Chukka, C. Coleman, S. Davis, P. Deng, G. Diamos, J. Duke, D. Fick, J. S. Gardner, I. Hubara, S. Idgunji, T. B. Jablin, J. Jiao, T. S. John, P. Kanwar, D. Lee, J. Liao, A. Lokhmotov, F. Massa, P. Meng, P. Micikevicius, C. Osborne, G. Pekhimenko, A. T. R. Rajan, D. Sequeira, A. Sirasao, F. Sun, H. Tang, M. Thomson, F. Wei, E. Wu, L. Xu, K. Yamada, B. Yu, G. Yuan, A. Zhong, P. Zhang, and Y. Zhou. 2020. MLPerf Inference Benchmark. In *ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. IEEE Press, Washington, DC, USA, 446–459. https://doi.org/10.1109/ISCA45697.2020.00045

[73] Scott Reed, Konrad Zolna, Emilio Parisotto, Sergio Gomez Colmenarejo, Alexander Novikov, Gabriel Barth-Maron, Mai Gimenez, Yury Sulsky, Jackie Kay, Jost Tobias Springenberg, Tom Eccles, Jake Bruce, Ali Razavi, Ashley Edwards, Nicolas Heess, Yutian Chen, Raia Hadsell, Oriol Vinyals, Mahyar Bordbar, and Nando de Freitas. 2022. A Generalist Agent. *Transactions on Machine Learning Research* 2022 (2022), 42 pages. https://openreview.net/forum?id=1ikK0kHjvj

[74] Kyle Roarty and Matthew D. Sinclair. 2020. Modeling Modern GPU Applications in gem5. In *3rd gem5 Users' Workshop*. 2 pages.

[75] Bita Rouhani, Daniel Lo, Ritchie Zhao, Ming Liu, Jeremy Fowers, Kalin Ovtcharov, Anna Vinogradsky, Sarah Massengill, Lita Yang, Ray Bittner, Alessandro Forin, Haishan Zhu, Taesik Na, Prerak Patel, Shuai Che, Lok Chand Koppaka, Xia Song, Subhojit Som, Kaustav Das, Saurabh Tiwary, Steve Reinhardt, Sitaram Lanka, Eric Chung, and Doug Burger. 2020. Pushing the Limits of Narrow Precision Inferencing at Cloud Scale with Microsoft Floating Point. In *Proceedings of the 34th International Conference on Neural Information Processing Systems* (Vancouver, BC, Canada) *(NeurIPS'20)*. Curran Associates Inc., Red Hook, NY, USA, Article 861, 11 pages.

[76] Aarush Selvan and Pankaj Kanwar. 2022. Google showcases Cloud TPU v4 Pods for large model training. https://cloud.google.com/blog/topics/tpus/google-showcases-cloud-tpu-v4-pods-for-large-model-training.

[77] Aashaka Shah, Vijay Chidambaram, Meghan Cowan, Saeed Maleki, Madan Musuvathi, Todd Mytkowicz, Jacob Nelson, Olli Saarikivi, and Rachee Singh. 2023. TACCL: Guiding Collective Algorithm Synthesis using Communication Sketches. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. USENIX Association, Boston, MA, 593–612. https://www.usenix.org/conference/nsdi23/presentation/shah

[78] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2019. Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism. *CoRR* abs/1909.08053 (2019), 9 pages. arXiv:1909.08053 [cs.CL] http://arxiv.org/abs/1909.08053

[79] Karen Simonyan and Andrew Zisserman. 2015. Very Deep Convolutional Networks for Large-Scale Image Recognition. In *3rd International Conference on Learning Representations (ICLR)*, Yoshua Bengio and Yann LeCun (Eds.). OpenReview.net, 14 pages. http://arxiv.org/abs/1409.1556

[80] Shaden Smith, Mostofa Patwary, Brandon Norick, Patrick LeGresley, Samyam Rajbhandari, Jared Casper, Zhun Liu, Shrimai Prabhumoye, George Zerveas, Vijay Korthikanti, Elton Zhang, Rewon Child, Reza Yazdani Aminabadi, Julie Bernauer, Xia Song, Mohammad Shoeybi, Yuxiong He, Michael Houston, Saurabh Tiwary, and Bryan Catanzaro. 2022. Using DeepSpeed and Megatron to Train Megatron-Turing NLG 530b, a Large-scale Generative Language Model. *arXiv preprint arXiv:2201.11990* (2022), 44 pages. arXiv:2201.11990 [cs.CL]

[81] Matthias Springer, Peter Wauligmann, and Hidehiko Masuhara. 2017. Modular Array-Based GPU Computing in a Dynamically-Typed Language. In *Proceedings of the 4th ACM SIGPLAN International Workshop on Libraries, Languages, and Compilers for Array Programming* (Barcelona, Spain) *(ARRAY 2017)*. Association for Computing Machinery, New York, NY, USA, 48–55. https://doi.org/10.1145/3091966.3091974

[82] Christian Szegedy, Wei Liu, Yangqing Jia, Pierre Sermanet, Scott Reed, Dragomir Anguelov, Dumitru Erhan, Vincent Vanhoucke, and Andrew Rabinovich. 2015. Going Deeper with Convolutions. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE Press, Piscataway, NJ, USA, 1–9.

[83] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jonathon Shlens, and Zbigniew Wojna. 2016. Rethinking the Inception Architecture for Computer Vision. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE Press, Piscataway, NJ, USA, 2818–2826. https://doi.org/10.1109/CVPR.2016.308

[84] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention Is All You Need. In *Proceedings of the 31st International Conference on Neural Information Processing Systems* (Long Beach, California, USA) *(NeurIPS)*. Curran Associates, Inc., Red Hook, NY, USA, 6000–6010. https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html

[85] Guibin Wang, YiSong Lin, and Wei Yi. 2010. Kernel Fusion: An Effective Method for Better Power Efficiency on Multithreaded GPU. In *Proceedings of the 2010 IEEE/ACM Int'l Conference on Green Computing and Communications & Int'l Conference on Cyber, Physical and Social Computing (GREENCOM-CPSCOM '10)*. IEEE Computer Society, USA, 344–350. https://doi.org/10.1109/GreenCom-CPSCom.2010.102

[86] Shibo Wang, Jinliang Wei, Amit Sabne, Andy Davis, Berkin Ilbeyi, Blake Hechtman, Dehao Chen, Karthik Srinivasa Murthy, Marcello Maggioni, Qiao Zhang, Sameer Kumar, Tongfei Guo, Yuanzhong Xu, and Zongwei Zhou. 2022. Overlap Communication with Dependent Computation via Decomposition in Large Deep Learning Models. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1 (ASPLOS)*. Association for Computing Machinery, New York, NY, USA, 93–106. https://doi.org/10.1145/3567955.3567959

[87] Wayne Xiong, , Xuedong Huang, Frank Seide, , and Andreas Stolcke. 2017. Toward Human Parity in Conversational Speech Recognition. *IEEE/ACM Transactions on Audio, Speech, and Language Processing* 25 (Sept 2017), 2410–2423.

Received 10 August 2023; revised 3 January 2024; accepted 8 January 2024